# Part-of-Speech (POS) Tagger for Hindi Language

CSI-5201 - Fundamentals of NLP

MSc (Artificial Intelligence) - Part 1 (Semester 1)

Mini Project - Implementation of NLP Problem

**Team Members:**
Muskan Khatoon (MSc_AI_P1_10)
Lisa Meave Luis (MSc_AI_P1_14)
Kowsya Mutkundu (MSc_AI_P1_17)
Muskan Aga (MSc_AI_P1_21)
Rochel Ria Da Silva (MSc_AI_P1_24)

October 31, 2025

# Contents


**1 Introduction** **5**

**2 Problem Statement** **5**

**3 Objectives** **5**

**4 Proposed Methodology** **5**
4.1 Data Collection . . . . . 5
4.2 Data Preprocessing . . . . . 6
4.3 Feature Extraction . . . . . 6

**5 Model Implementation** **6**

**6 Evaluation Metrics** **8**

**7 NLP Pipeline Overview** **8**

**8 Expected Outcomes** **8**

**9 Exploratory Data Analysis (EDA) and Preprocessing** **9**
9.1 Theory . . . . . 9
9.2 Importance of EDA and Preprocessing . . . . . 9
9.3 Implementation Snapshot . . . . . 10
9.4 Conclusion . . . . . 17

**10 Rule-Based POS Tagging** **18**
10.1 Introduction to Rule-Based Model . . . . . 18
10.2 Why Rule-Based POS Tagging for Hindi? . . . . . 18
10.3 Rule-Based POS Tagger Implementation in Google Colab . . . . . 19
10.3.1 1. Define Rule-Based POS Tagger . . . . . 19
10.3.2 2. Model Training and Prediction . . . . . 21
10.3.3 3. Model Evaluation . . . . . 22
10.3.4 4. Evaluation Results . . . . . 23
10.3.5 5. Sample Predictions . . . . . 24
10.3.6 6. Model Reload and Testing . . . . . 25
10.3.7 7. Gradio Interface for POS Tagging . . . . . 26
10.4 Advantages and Disadvantages of Rule-Based Tagger . . . . . 28
10.4.1 Advantages . . . . . 28
10.4.2 Disadvantages . . . . . 28
10.5 Conclusion . . . . . 28

**11 HMM** **29**
11.1 Introduction to HMM POS Tagger . . . . . 29
11.2 Why HMM for Hindi POS Tagging? . . . . . 29
11.3 HMM Implementation Steps in Google Colab . . . . . 29
11.3.1 1. Import Required Libraries . . . . . 29
11.3.2 2. Defining the `HMMPOSTagger` Class . . . . . 30

11.3.3 3. Initializing and Training the HMM POS Tagger . . . . . 31
11.3.4 4. Generating Predictions on Validation Data . . . . . 32
11.3.5 5. Model Evaluation using Accuracy and Classification Report . . 32
11.3.6 6. Displaying Sample Predictions from Validation Set . . . . . 33
11.3.7 7. Mounting Google Drive for Model Access . . . . . 34
11.3.8 8. Initialization and Training of HMM POS Tagger . . . . . 34
11.3.9 9. Implementation of HMM POS Tagger Class . . . . . 35
11.3.10 10. Extended HMM Implementation with Viterbi Algorithm . . . 35
11.3.11 11. Completion of Model Training . . . . . 36
11.3.12 12. Saving Model Parameters to Google Drive . . . . . 37
11.3.13 13. Deployment of HMM POS Tagger using Gradio Interface . . . 38
11.4 Advantages and Disadvantages of HMM . . . . . 40
11.4.1 Advantages . . . . . 40
11.4.2 Disadvantages . . . . . 40
11.5 Conclusion . . . . . 40

**12 CRF** **41**
12.1 Introduction to CRF . . . . . 41
12.2 Why CRF for Hindi POS Tagging? . . . . . 41
12.3 CRF Implementation Steps in Google Colab . . . . . 41
12.3.1 1. Set up Environment . . . . . 41
12.3.2 2. Mount Google Drive . . . . . 42
12.3.3 3. Load and Parse POS-tagged Files . . . . . 44
12.3.4 4. Feature Extraction . . . . . 44
12.3.5 5. Train/Test Split . . . . . 45
12.3.6 6. Train CRF Model . . . . . 46
12.3.7 7. Load Model & Predict . . . . . 47
12.3.8 8. Model Evaluation . . . . . 47
12.3.9 9. Tagging Custom Sentences . . . . . 48
12.3.10 10. Deployment Preparation on Hugging Face Spaces . . . . . 49
12.4 Advantages and Disadvantages of CRF . . . . . 50
12.4.1 Advantages . . . . . 50
12.4.2 Disadvantages . . . . . 50
12.5 Conclusion . . . . . 50

**13 BiLSTM** **51**
13.1 Introduction to BiLSTM . . . . . 51
13.2 Why BiLSTM for Hindi POS Tagging? . . . . . 51
13.3 Implementation Steps in Google Colab . . . . . 51
13.3.1 Step 1 — Import Required Libraries . . . . . 51
13.3.2 Step 2 — Data Preparation and Splitting . . . . . 51
13.3.3 Step 3 — Vocabulary Creation (Word and Tag Indexing) . . . . . 52
13.3.4 Step 4 — Build Final Word and Tag Vocabularies (from Training Data) . . . . . 53
13.3.5 Step 5 — Sequence Conversion and Padding . . . . . 53
13.3.6 Step 6 — Build and Compile the BiLSTM Model . . . . . 53
13.3.7 Step 7 — Train the BiLSTM Model . . . . . 54
13.3.8 Step 8 — Model Evaluation and Accuracy Calculation . . . . . 54

13.3.9 Step 9 — Testing Model Predictions on Sample Sentences . . . . 55
13.3.10 Step 10 — Save the Trained Model . . . . . 55
13.3.11 Step 11 — Install Required Deployment Libraries . . . . . 56
13.3.12 Step 12 — Save Vocabulary Mappings . . . . . 56
13.3.13 Step 13 — Load Model and Mappings for Deployment . . . . . 56
13.3.14 Step 14 — Define POS Prediction Function . . . . . 57
13.3.15 Step 15 — Deploy Hindi POS Tagger with Gradio . . . . . 57
13.4 Advantages and Disadvantages of BiLSTM . . . . . 57
13.4.1 Advantages . . . . . 57
13.4.2 Disadvantages . . . . . 58
13.5 Conclusion . . . . . 58

**14 mBERT Based Transformer Model for Hindi POS Tagging** **59**
14.1 Introduction . . . . . 59
14.2 Why mBERT for Hindi POS Tagging? . . . . . 59
14.3 Implementation Steps . . . . . 59
14.3.1 1. Environment Setup . . . . . 59
14.3.2 2. Mount Google Drive and Load Dataset . . . . . 59
14.3.3 3. Data Preprocessing . . . . . 60
14.3.4 4. Tokenization . . . . . 61
14.3.5 5. Model Training . . . . . 61
14.3.6 6. Evaluation . . . . . 63
14.3.7 7. Model Saving and Deployment . . . . . 63
14.3.8 8. Deployment of mBERT POS Tagger using Gradio Interface . . 64
14.4 Advantages of mBERT . . . . . 67
14.5 Disadvantages of mBERT . . . . . 67
14.6 Conclusion . . . . . 67

**15 Conclusion** **68**

**16 Model Deployments** **68**

**Abstract**

This project focuses on developing a Hindi Part-of-Speech (POS) tagger using multiple approaches including Rule-Based, Hidden Markov Model (HMM), Conditional Random Fields (CRF), BiLSTM, and Transformer-based models (mBERT / IndicBERT). POS tagging assigns each word in a sentence its grammatical category such as noun, verb, adjective, etc. While POS tagging in English has been well-studied, Hindi poses unique challenges due to its rich morphology, inflection, and free word order. This project collects pre-tagged Hindi datasets, trains and evaluates different models, and deploys the best-performing models through Hugging Face Spaces for real-time usage.

# 1 Introduction

Part-of-Speech (POS) tagging is a fundamental task in Natural Language Processing (NLP) that helps machines understand the grammatical structure of sentences. In Hindi, words change form depending on tense, gender, number, and case, making POS tagging a challenging task. Our project aims to create a POS tagger for Hindi by implementing five different approaches, evaluating their performance, and deploying them online for practical use. Each team member will focus on one model to ensure a comprehensive approach.

# 2 Problem Statement

Hindi POS tagging faces several challenges:

- Rich morphology: Words in Hindi change based on tense, gender, and number.
- Flexible word order: Hindi sentences can have multiple valid word arrangements.
- Limited annotated datasets compared to English.

Our goal is to build a robust POS tagging system that can accurately tag Hindi sentences and provide a tool for researchers and learners.

# 3 Objectives

1. To collect and preprocess a high-quality Hindi POS-tagged dataset.
2. To implement five POS tagging models: Rule-Based, HMM, CRF, BiLSTM, and Transformer (mBERT/IndicBERT).
3. To evaluate the models using accuracy, precision, recall, and F1-score.
4. To deploy the models via Hugging Face Spaces for real-time interaction.
5. To compare the performance of different models and analyze their strengths and weaknesses.

# 4 Proposed Methodology

The project follows a structured NLP pipeline as described below:

## 4.1 Data Collection

We will use pre-tagged Hindi datasets provided by our instructor. These datasets contain sentences with words already labeled with their respective POS tags. This saves time and ensures high-quality training data.

### 4.2 Data Preprocessing

Before training models, the data is cleaned and standardized:

- **Text Normalization:** Convert text to a consistent Unicode format and ensure uniform representation of words.
- **Tokenization:** Split sentences into individual words (tokens), handling punctuation and special characters carefully. Hindi-specific tokenizers are used for accuracy.

### 4.3 Feature Extraction

For traditional machine learning models (Rule-Based, HMM, CRF, SVM), we extract features such as:

- Word prefixes and suffixes
- Previous and next words in a sentence
- Surrounding POS tags (for sequential models)

For deep learning models (BiLSTM, Transformer):

- BiLSTM uses word embeddings to capture context in both directions.
- Transformers like mBERT/IndicBERT use pre-trained language models to understand contextual meaning with minimal feature engineering.

## 5 Model Implementation

In this project, we implemented five different POS tagging models for Hindi. Each model uses a different approach to predict the part-of-speech of each word in a sentence.

### 1. Rule-Based POS Tagger

Think of this as a set of "if-then" rules for tagging words.

**How it works:**

- If a word ends with "taa" $\rightarrow$ it's likely a verb.
- If a word ends with "ee" $\rightarrow$ it might be a feminine noun or adjective.
- It also checks a dictionary of known words.

**Analogy:** Like a teacher giving grammar rules to students. If the word looks like this, it gets this tag.

**Pros:** Simple, easy to understand. **Cons:** Hard to cover all cases; can make mistakes with new or unusual words.

### 2. Hidden Markov Model (HMM)

HMM is a probabilistic model that predicts a word's tag based on the previous word's tag.

**How it works:**

- Looks at sequences of tags, e.g., "Ram/NNP School/NN Jaata/V_VM Hai/V_VAUX".
- Calculates probabilities like: "After NNP, what is the most likely next tag?"
- Uses transition probabilities (tag → next tag) and emission probabilities (tag → word).

**Analogy:** Like predicting the next move in a chess game based on previous moves.

**Pros:** Handles sequences better than rule-based models. **Cons:** Only looks at limited context (usually one previous tag).

### 3. Conditional Random Fields (CRF)

CRF improves on HMM by looking at the whole sentence at once.

**How it works:**

- Considers the word itself, its neighbors, prefixes, suffixes, capitalization, and more.
- Predicts the most likely sequence of tags for the sentence.

**Analogy:** Like a teacher looking at the entire sentence before grading each word instead of judging words one by one.

**Pros:** Better accuracy; can include many features. **Cons:** Needs manual feature design; slower training on large datasets.

### 4. BiLSTM (Bidirectional Long Short-Term Memory)

BiLSTM is a deep learning model that reads the sentence both forward and backward.

**How it works:**

- Creates word embeddings (numerical representations of words).
- Predicts tags using context from both sides of the word.

**Analogy:** Like reading a sentence twice—once left-to-right and once right-to-left—to fully understand the meaning before deciding the word's role.

**Pros:** Learns features automatically; handles long sentences well. **Cons:** Needs a lot of data and more computing power.

### 5. Transformer-Based Models (mBERT / IndicBERT)

Transformers are state-of-the-art language models trained on huge text datasets.

**How it works:**

- Each word looks at all other words in the sentence at once using attention mechanisms.

- Fine-tuned on Hindi POS tagging (represented in transliteration) to predict tags accurately.

**Analogy:** Like having a super-smart language expert who can instantly understand the role of each word by seeing the whole sentence.

**Pros:** Very accurate; no manual feature engineering. **Cons:** Requires high computation and memory; slower on very large sentences.

## 6 Evaluation Metrics

All models are evaluated using:

- Accuracy: Percentage of correctly predicted POS tags.
- Precision: How many predicted tags are correct.
- Recall: How many actual tags were predicted correctly.
- F1-Score: Harmonic mean of precision and recall.
- Confusion Matrix: To identify specific tag-wise errors.

## 7 NLP Pipeline Overview

The full workflow of the project:

1. **Data Collection:** Gather pre-tagged Hindi datasets.
2. **Preprocessing:** Normalize text and tokenize sentences.
3. **Feature Extraction:** Extract features for traditional models; prepare embeddings for deep learning.
4. **Model Training:** Train Rule-Based, HMM, CRF, BiLSTM, and Transformer models.
5. **Evaluation:** Compute accuracy, precision, recall, F1-score, and analyze confusion matrices.
6. **Deployment:** Create Gradio interface in Colab and deploy models to Hugging Face Spaces.
7. **Monitoring and Updating:** Collect feedback, log errors, and retrain models if needed.

## 8 Expected Outcomes

- Comparative performance analysis of five models.
- Deployment of the models via Hugging Face Spaces for real-time POS tagging.
- Understanding of strengths, weaknesses, and computational efficiency of each model.
- A reusable pipeline for Hindi POS tagging research and applications.

# 9 Exploratory Data Analysis (EDA) and Preprocessing

## 9.1 Theory

Exploratory Data Analysis (EDA) is the process of examining datasets to summarize their main characteristics, often using visual methods. It helps in understanding the structure of the dataset, detecting anomalies, and discovering patterns before applying machine learning algorithms. In the context of Part-of-Speech (POS) tagging, EDA provides insights into the composition of the text data, such as the distribution of words, sentence lengths, and the variety of POS tags.

Preprocessing is a crucial step that prepares raw text data for model training. It includes operations such as data cleaning, normalization, tokenization, and structuring of word–tag pairs. Cleaning removes unwanted symbols or invisible Unicode characters, normalization ensures uniform text representation, and tokenization divides the text into meaningful units for further analysis.

In this project, the preprocessing pipeline involved reading multiple text files containing annotated sentences, extracting word–tag pairs, normalizing text using Unicode normalization, and splitting the dataset into training, validation, and testing subsets. This ensured that the model could learn efficiently from clean and balanced data while maintaining generalization capabilities across unseen text.

## 9.2 Importance of EDA and Preprocessing

EDA and preprocessing are vital for ensuring the quality and reliability of any Natural Language Processing (NLP) model. Without proper exploration and cleaning, datasets may contain noise, inconsistencies, or imbalances that can lead to poor model performance. EDA allows researchers to understand data distributions, detect outliers, and identify potential biases before model training.

Preprocessing ensures that the text data is standardized and machine-readable. It helps in removing irrelevant characters, managing inconsistent formatting, and handling linguistic variations. In the case of POS tagging, preprocessing ensures accurate mapping between words and their corresponding tags, which is critical for building robust tagging models and improving prediction accuracy.

### 9.3 Implementation Snapshot

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Figure 1: Mounting Google Drive to Access the Hindi Dataset in Google Colab

```
!ls /content/drive/MyDrive | head
```

```
ADT SPECIFICATION OF QUEUE.gdoc
ADT SPECIFICATION OF STACK.gdoc
archive (3).zip
archive.zip
Augmented Reality in Education: Understanding Its Impact .gform
Augmented Reality in Education: Understanding Its Impact  (Responses).gsheet
Book1.csv
Canva.gform
Certify'em Records.gsheet
chi_square_test_and_its_application_in_hypothesis.17.pdf
```

Figure 2: Listing the Contents of Google Drive to Verify Dataset Location

```
!ls /content/drive/MyDrive/HINDI_DATASET | head
```

```
hin_tourism_set01.txt
hin_tourism_set02.txt
hin_tourism_set03.txt
hin_tourism_set04.txt
hin_tourism_set05.txt
hin_tourism_set06.txt
hin_tourism_set07.txt
hin_tourism_set08.txt
hin_tourism_set09.txt
hin_tourism_set10.txt
```

Figure 3: Displaying the First Few Files from the HINDI DATASET Folder in Google Drive

```
data_folder = "/content/drive/MyDrive/HINDI_DATASET"
```

Figure 4: Defining the Path to the Hindi Dataset Directory Stored in Google Drive

```python
import os

sentences = []

for filename in os.listdir(data_folder):
    if filename.endswith(".txt"):
        file_path = os.path.join(data_folder, filename)
        with open(file_path, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue
                parts = line.split("\t")
                if len(parts) < 2:
                    continue
                tokens = parts[1].split()
                word_tag_pairs = []
                for token in tokens:
                    if "\\" in token:
                        word, tag = token.split("\\", 1)
                        word_tag_pairs.append((word, tag))
                sentences.append(word_tag_pairs)

print(f"✅ Loaded {len(sentences)} sentences.")
print("Example:", sentences[0][:10])
```

```
✅ Loaded 25025 sentences.
Example: []
```

Figure 5: Reading and Parsing Text Files from the Dataset Folder into Word–Tag Pairs

```python
import re, unicodedata

# Separate words and tags
all_words = [[w for w, t in sent] for sent in sentences]
all_tags = [[t for w, t in sent] for sent in sentences]

# Normalization function
def clean_text(text):
    text = unicodedata.normalize('NFKC', text)
    text = re.sub(r'[“”‘’]', '"', text)
    text = re.sub(r'[\u200c\u200d]', '', text)  # remove zero-width chars
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Apply cleaning
cleaned_sentences = [[clean_text(w) for w in s] for s in all_words]

print(f"✅ Cleaned {len(cleaned_sentences)} sentences.")
print("Example cleaned sentence:", cleaned_sentences[0])
```

```
✅ Cleaned 25025 sentences.
Example cleaned sentence: []
```

Figure 6: Text Normalization and Cleaning Process Applied to Sentences

```python
import numpy as np
from collections import Counter

num_sentences = len(cleaned_sentences)
num_tokens = sum(len(s) for s in cleaned_sentences)
unique_words = sorted({w for s in cleaned_sentences for w in s})
unique_tags = sorted({t for ts in all_tags for t in ts})

print(f"📈 Total Sentences: {num_sentences}")
print(f"🔡 Total Tokens: {num_tokens}")
print(f"🆕 Unique Words: {len(unique_words)}")
print(f"🏷️ Unique POS Tags: {len(unique_tags)}")
print("POS Tags List:", unique_tags)
```

```
📈 Total Sentences: 25025
🔡 Total Tokens: 424087
🆕 Unique Words: 28579
🏷️ Unique POS Tags: 112
POS Tags List: ['', 'CCC_CD', 'CC_CCD', 'CC_CCS', 'CC_CS', 'C_CCD', 'C_CCS', 'DD_DMD', 'DM_DM', 'DM_DMD', 'DM_DMI', 'DM_DMM', 'DM_DMQ'
```

Figure 7: Computation of Basic Statistical Information from the Dataset

```
import matplotlib.pyplot as plt
%matplotlib inline

tag_counts = Counter([t for ts in all_tags for t in ts])
plt.figure(figsize=(20,5))
plt.bar(tag_counts.keys(), tag_counts.values(), color='skyblue', edgecolor='black')
plt.xticks(rotation=90)
plt.title("POS Tag Frequency in Hindi Dataset")
plt.xlabel("POS Tag")
plt.ylabel("Count")
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 2404 (\N{DEVANAGARI DANDA}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Matplotlib currently does not support Devanagari natively.
  fig.canvas.print_figure(bytes_io, **kw)
```

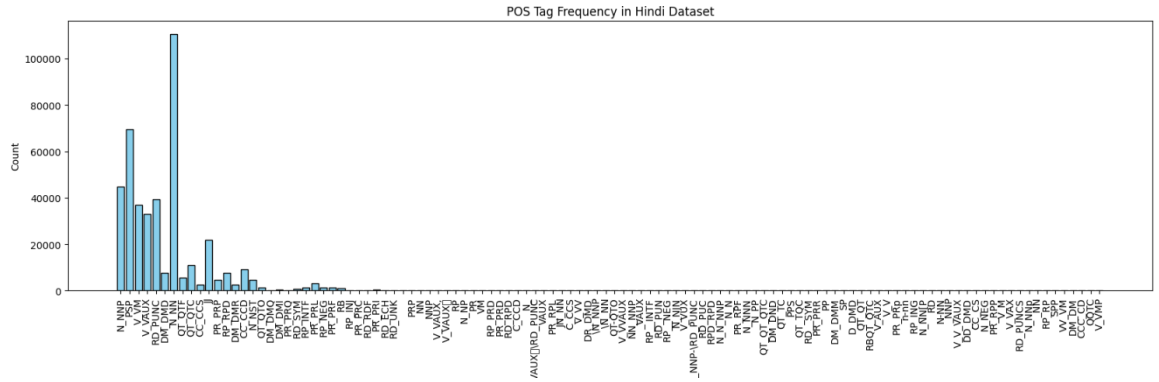
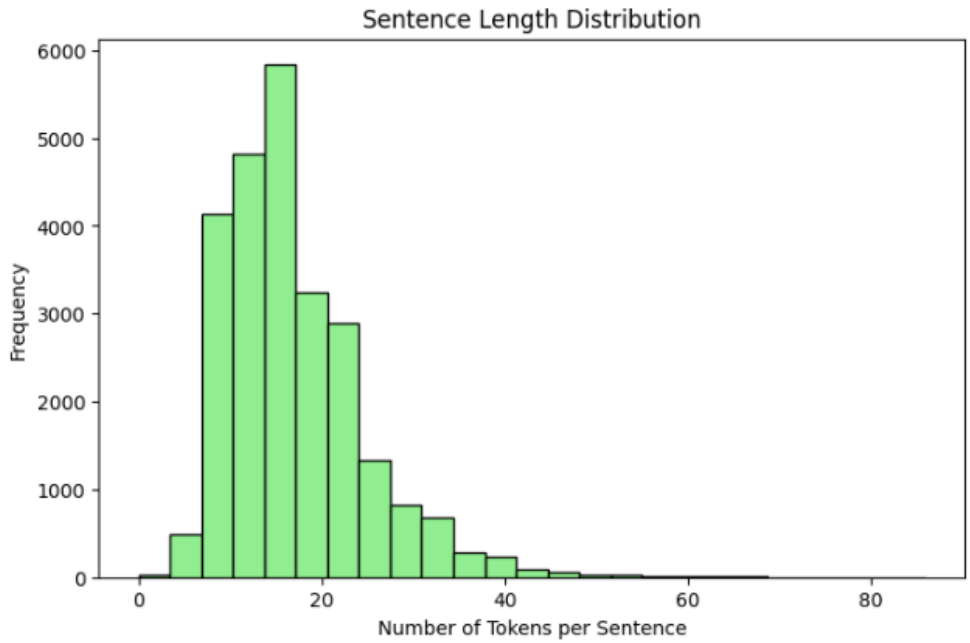



Figure 8: Visualization of POS Tag Frequency Distribution in the Dataset

```python
sent_lengths = [len(s) for s in cleaned_sentences]

plt.figure(figsize=(8,5))
plt.hist(sent_lengths, bins=25, color='lightgreen', edgecolor='black')
plt.title("Sentence Length Distribution")
plt.xlabel("Number of Tokens per Sentence")
plt.ylabel("Frequency")
plt.show()

print(f"Average Sentence Length: {np.mean(sent_lengths):.2f}")
print(f"Max Sentence Length: {np.max(sent_lengths)}")
print(f"Min Sentence Length: {np.min(sent_lengths)}")
```




```
Average Sentence Length: 16.95
Max Sentence Length: 86
Min Sentence Length: 0
```

Figure 9: Histogram Showing the Distribution of Sentence Lengths in the Dataset

```python
word_counts = Counter([w for s in cleaned_sentences for w in s])
print("🔝 Top 20 Frequent Words:")
for word, count in word_counts.most_common(20):
    print(f"{word:<20} {count}")

print("\n🏷️ Top 10 Frequent POS Tags:")
for tag, count in tag_counts.most_common(10):
    print(f"{tag:<10} {count}")
```

```
🔝 Top 20 Frequent Words:
।                    24453
के                   15941
है                   15718
में                  12663
की                    9824
                      9585
से                    9193
हैं                   7374
का                    7272
```

Figure 10: List of the Most Frequent Words and POS Tags in the Dataset”

Figure 11: Splitting the dataset into training, validation, and testing sets for model development

### 9.4 Conclusion

The EDA revealed valuable insights into the dataset's structure, such as the total number of sentences, token count, unique word and tag distributions, and sentence length variations. Visualizations like POS tag frequency and sentence length histograms provided a clear understanding of data balance.

The preprocessing steps effectively cleaned and standardized the dataset, removed inconsistencies, and prepared it for model training. As a result, the processed data formed a strong foundation for building accurate and robust POS tagging models in subsequent stages.

# 10 Rule-Based POS Tagging

## 10.1 Introduction to Rule-Based Model

**In simple terms:**

- Rule-based POS tagging assigns tags based on predefined linguistic rules and known word-tag pairs.
- It uses word frequency (from training data) and pattern-based rules such as suffixes and postpositions.
- It is simple, interpretable, and useful when the dataset is small or limited.

## 10.2 Why Rule-Based POS Tagging for Hindi?

Hindi is a morphologically rich language, meaning many words are formed using various suffixes and inflections. Challenges include:

- Same root word can appear in different forms depending on tense, number, or gender.

Rule-based tagging is suitable for Hindi because:

- Linguistic rules (such as suffix patterns) can effectively identify nouns, verbs, or postpositions.
- It is fast and interpretable, providing a foundation for understanding more complex models.
- Works well for small datasets and in the absence of large annotated corpora.

### 10.3 Rule-Based POS Tagger Implementation in Google Colab

Below are the step-by-step stages carried out to build the Hindi Rule-Based POS Tagger.

#### 10.3.1 1. Define Rule-Based POS Tagger

- A class named **RuleBasedPOSTagger** is created to perform POS tagging for Hindi text using frequency-based learning and simple linguistic rules.
- It initializes:
  - **word_tag_freq**: Stores word–tag frequency counts using `defaultdict(Counter)`.
  - **rules**: A list of regex-based suffix and pattern rules for unseen words.
  - **default_tag**: Set as `"UNK"` for unknown words.
- The class also includes methods to train on data, predict tags, and save or load the model using `pickle`.

Rule-Based POS Tagging

```python
# model.py (inline in Colab)
import re, pickle
from collections import defaultdict, Counter

class RuleBasedPOSTagger:
    def __init__(self):
        self.word_tag_freq = defaultdict(Counter)
        self.rules = []
        self.default_tag = "UNK"

    def train(self, sentences, tags):
        for s, t in zip(sentences, tags):
            for word, tag in zip(s, t):
                self.word_tag_freq[word][tag] += 1

        self.rules = [
            (re.compile(r"(ा|ी|ें|ों)$"), "N_NN"),
            (re.compile(r"(ना|ती|ता|एगा|ेगी|ेगा|ते हैं)$"), "V_VM"),
            (re.compile(r"(से|पर|में|के|का|की|को)$"), "PSP"),
            (re.compile(r"^[0-9]+$"), "QC"),
            (re.compile(r"[।,!?;:.]$"), "RD_PUNC"),
        ]
```

Figure 12: Defining the Rule-Based POS Tagger class and its training and prediction logic

```
def predict(self, sentence):
    tags = []
    for word in sentence:
        if word in self.word_tag_freq:
            tag = self.word_tag_freq[word].most_common(1)[0][0]
        else:
            tag = self.default_tag
            for pattern, rule_tag in self.rules:
                if pattern.search(word):
                    tag = rule_tag
                    break
        tags.append(tag)
    return tags

def save(self, path):
    with open(path, "wb") as f:
        pickle.dump(self, f)

@staticmethod
def load(path):
    with open(path, "rb") as f:
        return pickle.load(f)
```

Figure 13: Defining the Rule-Based POS Tagger class and its training and prediction logic

### 10.3.2 2. Model Training and Prediction

- An instance of **RuleBasedPOSTagger** is created and trained using the training sentences and their corresponding tags.
- The trained model is saved as `model.pkl` for later use.
- For evaluation, the model predicts tags on the validation dataset, and the true and predicted tags are stored in separate lists (`y_true` and `y_pred`) for accuracy calculation.

```
# Initialize and train
rule_tagger = RuleBasedPOSTagger()
rule_tagger.train(train_sents, train_tags)
rule_tagger.save("model.pkl")
print("✅ Model saved as model.pkl")

# Predict on validation set
y_true, y_pred = [], []
for s, t in zip(val_sents, val_tags):
    pred = rule_tagger.predict(s)
    y_true.extend(t)
    y_pred.extend(pred)
```

```
✅ Model saved as model.pkl
```

Figure 14: Training the Rule-Based POS Tagger and predicting tags on the validation dataset

### 10.3.3 3. Model Evaluation

- The model's performance is evaluated using **accuracy score** and a detailed **classification report** from the `sklearn.metrics` module.
- These metrics provide insights into the overall accuracy and tag-wise precision, recall, and F1-score of the Rule-Based POS Tagger.

```
from sklearn.metrics import classification_report, accuracy_score

print("✅ Rule-Based POS Tagger Evaluation:")
print("Accuracy:", accuracy_score(y_true, y_pred))
print("\nDetailed Report:\n")
print(classification_report(y_true, y_pred, zero_division=0))
```

```
✅ Rule-Based POS Tagger Evaluation:
Accuracy: 0.8279681143743974

Detailed Report:

              precision    recall  f1-score   support

      CC_CCD       0.94      0.95      0.94       923
      CC_CCS       0.73      0.77      0.75       281
      DM_DMD       0.69      0.87      0.77       784
      DM_DMI       0.55      0.85      0.67        66
      DM_DMQ       0.25      0.12      0.17        16
      DM_DMR       0.34      0.15      0.20       234
          JJ       0.75      0.71      0.73      2168
        N-NN       0.00      0.00      0.00         1
      NN_NNP       0.00      0.00      0.00         1
         N_N       0.00      0.00      0.00         1
        N_NN       0.84      0.83      0.84     11268
```

Figure 15: Evaluating the Rule-Based POS Tagger using accuracy and classification metrics

### 10.3.4 4. Evaluation Results

- The Rule-Based POS Tagger achieved an overall accuracy of **82.79%**.
- High performance was observed for frequent tags like `N_NN`, `PSP`, and `RD_PUNC`.
- Lower scores for rare tags indicate limited training data and fewer rule matches.
- Overall, the model shows effective tagging accuracy for common Hindi word categories.

| | | | | |
|---|---|---|---|---|
| accuracy | | | 0.83 | 42527 |
| macro avg | 0.48 | 0.46 | 0.45 | 42527 |
| weighted avg | 0.85 | 0.83 | 0.84 | 42527 |

Figure 16: Evaluation Results

### 10.3.5 5. Sample Predictions

- A few validation sentences are displayed along with their true and predicted POS tags.
- This helps visually compare model predictions with actual annotations to assess tagging accuracy.

```
for i in range(3):
    print("Sentence:", " ".join(val_sents[i]))
    print("True Tags:", val_tags[i])
    print("Predicted:", rule_tagger.predict(val_sents[i]))
    print("-"*80)
```

```
Sentence: इस कला महोत्सव का मुख्य मकसद विभिन्न देशों के कलाविदों के बीच सांस्कृतिक व कलात्मक संवाद विकसित करना था ।
True Tags: ['DM_DMD', 'N_NN', 'N_NN', 'PSP', 'JJ', 'N_NN', 'JJ', 'N_NN', 'PSP', 'N_NN', 'PSP', 'N_NST',
Predicted: ['DM_DMD', 'N_NN', 'N_NN', 'PSP', 'JJ', 'N_NN', 'JJ', 'N_NN', 'PSP', 'N_NN', 'PSP', 'N_NST',
--------------------------------------------------------------------------------
Sentence: मकाऊ में जेटी से बाहर निकलते ही रिक्शे पर ऊँघते रिक्शा चालकों को देख कर अजीब सा सुखद एहसास होता है ।
True Tags: ['N_NNP', 'PSP', 'N_NN', 'PSP', 'N_NST', 'V_VM', 'RP_RPD', 'N_NN', 'PSP', 'V_VM', 'N_NN', 'N
Predicted: ['N_NNP', 'PSP', 'N_NN', 'PSP', 'N_NST', 'V_VM', 'RP_RPD', 'N_NN', 'PSP', 'UNK', 'N_NN', 'N_I
--------------------------------------------------------------------------------
Sentence: इसी से मिलती - जुलती स्केटिंग है सिंक्रेनाइज्ड स्केटिंग ।
True Tags: ['PR_PRL', 'PSP', 'V_VM', 'RD_PUNC', 'RD_ECH', 'N_NN', 'V_VM', 'N_NN', 'N_NN', 'RD_PUNC']
Predicted: ['DM_DMD', 'PSP', 'V_VM', 'RD_PUNC', 'N_NN', 'N_NN', 'V_VAUX', 'UNK', 'N_NN', 'RD_PUNC']
--------------------------------------------------------------------------------
```

Figure 17: Sample Predictions

### 10.3.6 6. Model Reload and Testing

- The saved model (`model.pkl`) is reloaded using the `load()` method.
- A sample Hindi sentence is tested to verify that the model correctly loads and predicts POS tags without retraining.

```
# quick reload + test
model_test = RuleBasedPOSTagger.load("model.pkl")
sample = ["मैं", "स्कूल", "जा", "रहा", "हूँ", "।"]
print("Sentence:", sample)
print("Predicted Tags:", model_test.predict(sample))
```

```
Sentence: ['मैं', 'स्कूल', 'जा', 'रहा', 'हूँ', '।']
Predicted Tags: ['PR_PRP', 'N_NN', 'V_VAUX', 'V_VAUX', 'V_VAUX', 'RD_PUNC']
```

Figure 18: Model Reload and Testing

### 10.3.7 7. Gradio Interface for POS Tagging

- A simple **Gradio** interface is created to interact with the trained model.
- Users can input a Hindi sentence, and the app displays each word with its predicted POS tag.
- The model is loaded from `model.pkl`, and the interface is launched using `share=True` for easy access from Google Colab.

```
import gradio as gr

# Load model
tagger = RuleBasedPOSTagger.load("model.pkl")

def tokenize_hindi(text):
    return text.strip().split()

def predict_tags(text):
    tokens = tokenize_hindi(text)
    tags = tagger.predict(tokens)
    return "\n".join(f"{w} / {t}" for w, t in zip(tokens, tags))

with gr.Blocks() as demo:
    gr.Markdown("# IN Hindi Rule-Based POS Tagger")
    gr.Markdown("Enter a Hindi sentence below to get POS tags.")
    inp = gr.Textbox(lines=4, placeholder="यहाँ वाक्य लिखें...")
    out = gr.Textbox(lines=8)
    btn = gr.Button("Tag Sentence")
    btn.click(fn=predict_tags, inputs=inp, outputs=out)

demo.launch(share=True)  # use share=True to test it directly from Colab
```

Figure 19: Gradio Interface for POS Tagging

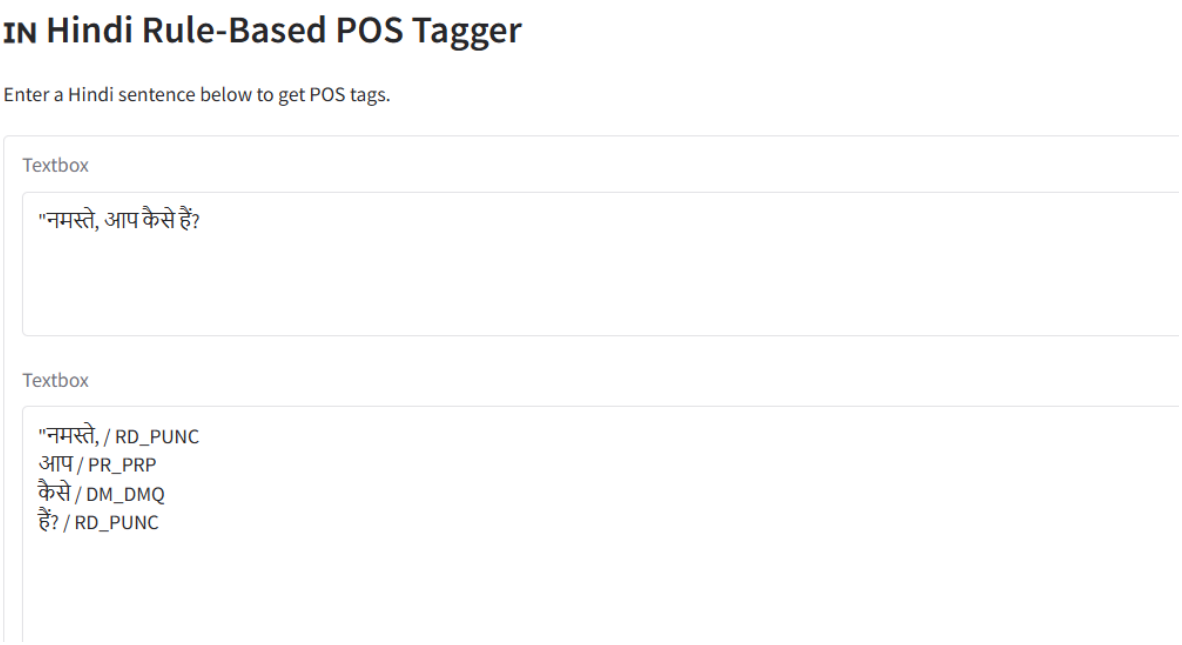



Figure 20: Gradio Interface for POS Tagging

## 10.4 Advantages and Disadvantages of Rule-Based Tagger

### 10.4.1 Advantages

- Simple, interpretable, and fast to implement.
- Works well with small datasets or low-resource languages.
- Captures basic linguistic knowledge effectively.

### 10.4.2 Disadvantages

- Requires manual rule creation — time-consuming and language-specific.
- Struggles with unseen or ambiguous words.
- Lacks context awareness; decisions are mostly local.

**Reason to move to next model:** Since rule-based models cannot automatically learn contextual or long-distance dependencies, we moved on to probabilistic and statistical models such as HMM for improved performance.

## 10.5 Conclusion

The Rule-Based POS Tagger provided a strong baseline for Hindi POS tagging:

- Implemented simple linguistic rules for suffix and postposition detection.
- Achieved good accuracy for known and rule-matching words.
- Served as a foundation for understanding challenges that later models like HMM and CRF solve automatically.
- The model was successfully deployed online for demonstration using Gradio and Hugging Face Spaces.

# 11 HMM

## 11.1 Introduction to HMM POS Tagger

Hidden Markov Model (HMM) is a generative sequence model commonly used for tasks such as Part-of-Speech (POS) tagging. In the code provided, the HMM is implemented with explicit transition and emission probability tables and a Viterbi decoder for inference.

**In simple terms:**

- HMM models the joint probability of a sequence of tags and the observed words by using *transition* probabilities between tags and *emission* probabilities of words given tags.
- Decoding (predicting tags for a sentence) is done with the Viterbi algorithm which finds the most probable tag sequence.

## 11.2 Why HMM for Hindi POS Tagging?

Hindi is morphologically rich and often benefits from a structured, probabilistic sequence model. The HMM implementation in your code is suitable when:

- You want a simple, interpretable baseline that explicitly models tag-to-tag transitions and tag-to-word emissions.
- You need a fast, classical approach (training is count-based with smoothing) before trying more complex discriminative or neural sequence models.

## 11.3 HMM Implementation Steps in Google Colab

Below are the detailed steps carried out in Colab, written to match the exact operations performed by your code.

### 11.3.1 1. Import Required Libraries

- Imports essential Python modules required for building the HMM POS Tagger.
- `defaultdict` and `Counter` from the `collections` module help store frequency counts and default values efficiently.
- `defaultdict` will later be used to create nested dictionaries for transition and emission probabilities without key errors.
- `Counter` is useful for counting the occurrences of tags and words in the training data.
- `numpy` (`np`) is imported for future numerical operations that may be required during calculations.
- This cell focuses only on importing required dependencies — no model logic is implemented here.

```
from collections import defaultdict, Counter
import numpy as np
```

Figure 21: Importing the required Python libraries for building the HMM POS Tagger.

### 11.3.2 2. Defining the `HMMPOSTagger` Class

- This cell defines the main class `HMMPOSTagger`, which encapsulates all logic for training and predicting POS tags using a Hidden Markov Model.
- The `__init__()` method initializes data structures needed to store transition probabilities, emission probabilities, vocabulary, and tags.
- `transition_probs` and `emission_probs` use nested `defaultdict` to avoid key errors and to dynamically store probability values.
- `start_tag` and `end_tag` are defined to mark sentence boundaries, helping the model learn how sentences begin and end.
- The `train()` method calculates frequency counts of tag transitions and word-tag pairs, and then converts these counts into probabilities using Add-1 smoothing.
- **Add-1 smoothing** ensures that probabilities never become zero, even for unseen transitions or words during training.
- The `viterbi()` method implements the Viterbi Algorithm, which uses dynamic programming to find the most likely POS tag sequence for a given sentence.
- It first initializes probabilities for the first word, then iteratively computes the best sequence of tags for the remaining words, finally returning the optimal tag path.
- This single class contains both the training and decoding (prediction) mechanisms required for the HMM POS Tagger.

```python
class HMMPOSTagger:
    def __init__(self):
        self.transition_probs = defaultdict(lambda: defaultdict(float))
        self.emission_probs = defaultdict(lambda: defaultdict(float))
        self.tags = set()
        self.vocab = set()
        self.tag_counts = Counter()
        self.start_tag = "<s>"
        self.end_tag = "</s>"

    def train(self, sentences, tags):
        # Count tag and word occurrences
        transition_counts = defaultdict(Counter)
        emission_counts = defaultdict(Counter)

        for s, t in zip(sentences, tags):
            prev_tag = self.start_tag
            for word, tag in zip(s, t):
                transition_counts[prev_tag][tag] += 1
                emission_counts[tag][word] += 1
                self.tag_counts[tag] += 1
                self.tags.add(tag)
                self.vocab.add(word)
                prev_tag = tag
            transition_counts[prev_tag][self.end_tag] += 1

        # Compute probabilities (add-1 smoothing)
        for prev_tag in transition_counts:
            total = sum(transition_counts[prev_tag].values()) + len(self.tags)
            for tag in self.tags.union({self.end_tag}):
                self.transition_probs[prev_tag][tag] = (transition_counts[prev_tag][tag] + 1) / total

        for tag in emission_counts:
            total = sum(emission_counts[tag].values()) + len(self.vocab)
            for word in self.vocab:
                self.emission_probs[tag][word] = (emission_counts[tag][word] + 1) / total

    def viterbi(self, sentence):
        V = [{}]
        path = {}

        # Initialize base cases (start tag)
        for tag in self.tags:
```

Figure 22: Definition of the `HMMPOSTagger` class containing training and Viterbi decoding logic.

### 11.3.3 3. Initializing and Training the HMM POS Tagger

- An object of the class `HMMPOSTagger` is created and assigned to the variable `hmm_tagger`.
- This initializes all required internal data structures such as transition tables, emission tables, vocabulary set, and tag set.
- The `train()` function is then called using the training sentences (`train_sents`) and their corresponding POS tag labels (`train_tags`).
- During training, the model learns:
  - How likely one tag follows another (transition probabilities).
  - How likely a word is associated with a tag (emission probabilities).
- The printed message " `HMM training completed!`" confirms that the training phase has successfully finished without errors.
- At the end of this cell, the HMM model becomes fully prepared for POS tag prediction on new/unseen sentences.

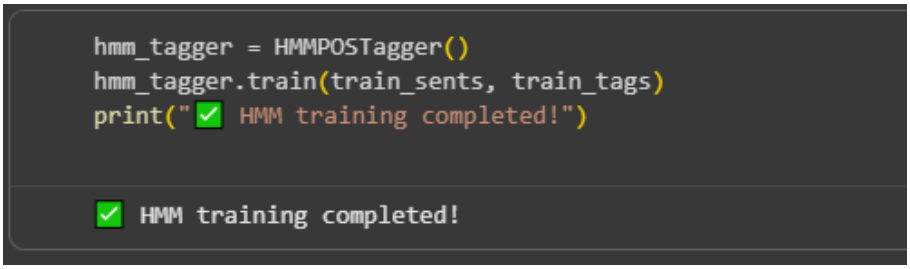



Figure 23: Initializing the HMMPOSTagger object and training it on labelled data.

### 11.3.4 4. Generating Predictions on Validation Data

- Two empty lists, `y_true` and `y_pred`, are initialized to store the actual POS tags and the predicted POS tags respectively.
- A loop is executed over each sentence (`s`) and its corresponding tag sequence (`t`) from the validation dataset (`val_sents` and `val_tags`).
- A condition is included to skip any empty sentences or tag lists to avoid processing errors.
- For each valid sentence, the trained model's `viterbi()` method is called to predict the most likely POS tag sequence.
- The original tags are added to `y_true`, and the model's predicted tags are added to `y_pred` for later evaluation.
- By the end of this cell, two aligned lists are prepared — one with true labels and one with predicted labels — enabling accurate model performance calculation.

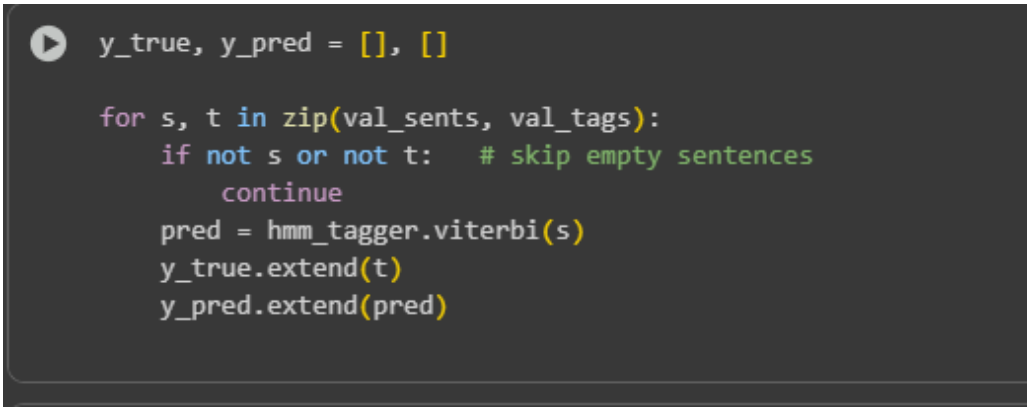



Figure 24: Predicting POS tags for validation sentences and storing true vs. predicted labels.

### 11.3.5 5. Model Evaluation using Accuracy and Classification Report

- This step imports two key evaluation metrics from `sklearn.metrics`:
  - `accuracy_score` – measures the overall percentage of correctly predicted POS tags.

– `classification_report` – provides precision, recall, F1-score for each tag.

- A header message is printed to indicate the start of the evaluation phase for the HMM POS Tagger.
- The `accuracy_score(y_true, y_pred)` function computes how many predicted tags match the actual tags from the validation set.
- The `classification_report` displays a detailed table showing performance per POS tag, helping to analyze which tags the model learned well and which need improvement.
- Parameter `zero_division=0` ensures that if any POS tag has 0 predictions, the report will not throw an error and instead records the metric as 0.
- This cell completes the evaluation stage and provides both a high-level and detailed performance overview of the trained HMM model.

```
from sklearn.metrics import classification_report, accuracy_score

print("✅ HMM POS Tagger Evaluation:")
print("Accuracy:", accuracy_score(y_true, y_pred))
print("\nDetailed Report:\n")
print(classification_report(y_true, y_pred, zero_division=0))
```

```
✅ HMM POS Tagger Evaluation:
Accuracy: 0.810967150280998

Detailed Report:

              precision    recall  f1-score   support

       CC_CCD       0.94      0.93      0.94       923
       CC_CCS       0.67      0.70      0.68       281
       DM_DMD       0.65      0.90      0.75       784
       DM_DMI       0.56      0.33      0.42        66
       DM_DMQ       0.00      0.00      0.00        16
       DM_DMR       0.20      0.00      0.01       234
           JJ       0.67      0.68      0.67      2168
         N-NN       0.00      0.00      0.00         1
       NN_NNP       0.00      0.00      0.00         1
          N_N       0.00      0.00      0.00         1
         N_NN       0.79      0.82      0.81     11268
        N_NNN       0.00      0.00      0.00         1
        N_NNP       0.78      0.67      0.72      4356
         N_NP       0.00      0.00      0.00         1
        N_NST       0.77      0.43      0.55       510
       PR_PRC       0.00      0.00      0.00         1
       PR_PRF       0.98      0.90      0.94       155
       PR_PRI       0.00      0.00      0.00        49
       PR_PRL       0.82      0.72      0.77       329
       PR_PRP       0.85      0.80      0.82       463
       PR_PRQ       0.50      0.05      0.09        20
       PR_PRR       0.00      0.00      0.00         1
       PR_RPL       0.00      0.00      0.00         1
       PR_RPP       0.00      0.00      0.00         1
          PSP       0.89      0.98      0.93      6918
        QT_QT       0.00      0.00      0.00         1
       QT_QTC       0.91      0.79      0.85      1044
       QT_QTF       0.68      0.54      0.60       583
       QT_QTO       0.90      0.20      0.32        96
           RB       1.00      0.01      0.02       106
       RD_ECH       0.00      0.00      0.00        31
      RD_PUNC       0.97      0.97      0.97      4024
       RD_RDF       0.00      0.00      0.00         9
       RD_SYM       0.43      0.28      0.34        58
```

Figure 25: Evaluating the HMM POS Tagger using accuracy and detailed metrics report.

### 11.3.6 6. Displaying Sample Predictions from Validation Set

- This code snippet prints a few sample outputs to visually inspect how well the HMM model is tagging sentences.
- A loop runs for the first three sentences from the validation dataset (`val_sents` and `val_tags`).

- For each sentence:
  - The sentence is printed in readable text form using `" ".join(val_sents[i])`.
  - The actual (gold) POS tags are displayed for comparison.
  - The HMM model's predicted POS tags for the same sentence are obtained using the `viterbi()` method.
- A separator line of hyphens (`"-" * 80`) is printed after each sample to improve readability of outputs.
- This cell allows a quick qualitative check to see whether predictions align with the true tags before deeper evaluation analysis.

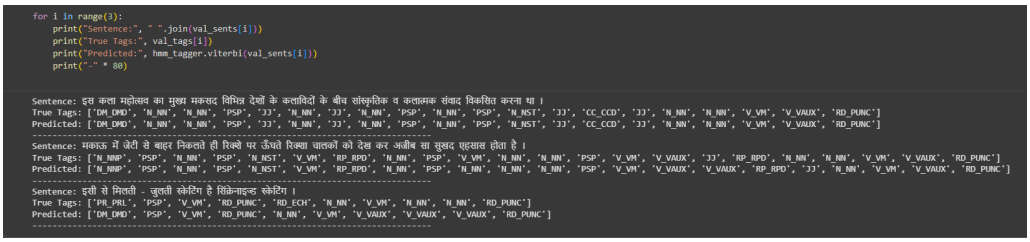



Figure 26: Sample comparison of true vs predicted POS tags for validation sentences.

### 11.3.7 7. Mounting Google Drive for Model Access

- Mounted Google Drive in Colab to access datasets and save trained model files.
- Ensured seamless integration between local Colab runtime and persistent cloud storage.
- Facilitated easy loading and saving of intermediate outputs and model checkpoints.

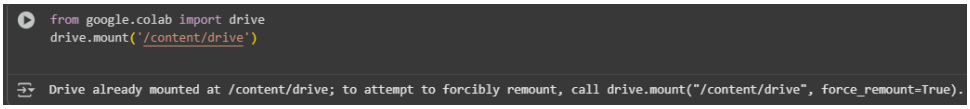



Figure 27: Google Drive mounted successfully in Colab environment, allowing persistent storage for training data and model artifacts.

### 11.3.8 8. Initialization and Training of HMM POS Tagger

- Initialized the HMM POS tagger object and trained it using preprocessed Hindi data.
- Computed transition and emission probabilities based on word-tag pairs.
- This marks the start of the model training phase for sequence labeling.

```python
hmm_tagger = HMMPOSTagger()
hmm_tagger.train(train_sents, train_tags)
```

Figure 28: Code snippet showing initialization and training of the HMM POS tagger using tokenized Hindi sentences and corresponding tags.

### 11.3.9 9. Implementation of HMM POS Tagger Class

- Implemented the `HMMPOS_Tagger` class, defining methods for training and probability calculation.
- Utilized `defaultdict` and `Counter` to manage frequency counts efficiently.
- Added Laplace (add-one) smoothing to handle unseen transitions and emissions.

```python
from collections import defaultdict, Counter
import numpy as np

def float_defaultdict():
    return defaultdict(float)

class HMMPOSTagger:
    def __init__(self):
        self.transition_probs = defaultdict(float_defaultdict)
        self.emission_probs = defaultdict(float_defaultdict)
        self.tags = set()
        self.vocab = set()
        self.tag_counts = Counter()
        self.start_tag = "<s>"
        self.end_tag = "</s>"

    def train(self, sentences, tags):
        transition_counts = defaultdict(Counter)
        emission_counts = defaultdict(Counter)

        for s, t in zip(sentences, tags):
            prev_tag = self.start_tag
            for word, tag in zip(s, t):
                transition_counts[prev_tag][tag] += 1
                emission_counts[tag][word] += 1
                self.tag_counts[tag] += 1
                self.tags.add(tag)
                self.vocab.add(word)
                prev_tag = tag
            transition_counts[prev_tag][self.end_tag] += 1

        # Compute probabilities (add-1 smoothing)
        for prev_tag in transition_counts:
            total = sum(transition_counts[prev_tag].values()) + len(self.tags)
            for tag in self.tags.union({self.end_tag}):
                self.transition_probs[prev_tag][tag] = (transition_counts[prev_tag][tag] + 1) / total

        for tag in emission_counts:
            total = sum(emission_counts[tag].values()) + len(self.vocab)
            for word in self.vocab:
                self.emission_probs[tag][word] = (emission_counts[tag][word] + 1) / total
```

Figure 29: Implementation of the `HMMPOS_Tagger` class with functions to estimate transition and emission probabilities from tagged corpora.

### 11.3.10 10. Extended HMM Implementation with Viterbi Algorithm

- Enhanced HMM tagger by defining the Viterbi decoding algorithm.
- Introduced start (`<s>`) and end (`</s>`) tags to define sentence boundaries.

- The Viterbi method helps determine the most likely POS sequence for unseen data.

```python
from collections import defaultdict, Counter
import numpy as np

# ✅ define helper function to replace lambda
def float_defaultdict():
    return defaultdict(float)

class HMMPOSTagger:
    def __init__(self):
        self.transition_probs = defaultdict(float_defaultdict)
        self.emission_probs = defaultdict(float_defaultdict)
        self.tags = set()
        self.vocab = set()
        self.tag_counts = Counter()
        self.start_tag = "<s>"
        self.end_tag = "</s>"

    def train(self, sentences, tags):
        transition_counts = defaultdict(Counter)
        emission_counts = defaultdict(Counter)

        for s, t in zip(sentences, tags):
            prev_tag = self.start_tag
            for word, tag in zip(s, t):
                transition_counts[prev_tag][tag] += 1
                emission_counts[tag][word] += 1
                self.tag_counts[tag] += 1
                self.tags.add(tag)
                self.vocab.add(word)
                prev_tag = tag
            transition_counts[prev_tag][self.end_tag] += 1

        for prev_tag in transition_counts:
            total = sum(transition_counts[prev_tag].values()) + len(self.tags)
            for tag in self.tags.union({self.end_tag}):
                self.transition_probs[prev_tag][tag] = (transition_counts[prev_tag][tag] + 1) / total

        for tag in emission_counts:
            total = sum(emission_counts[tag].values()) + len(self.vocab)
            for word in self.vocab:
                self.emission_probs[tag][word] = (emission_counts[tag][word] + 1) / total

    def viterbi(self, sentence):
        V = [{}]
        path = {}
```

Figure 30: Extended implementation of the HMM tagger with initialization functions, transition and emission probability calculations, and the Viterbi decoding logic.

### 11.3.11 11. Completion of Model Training

- Successfully trained the HMM POS tagger model on the Hindi dataset.
- Verified training completion using status output and Colab cell indicators.
- The trained model is now ready for evaluation or serialization.

```python
hmm_tagger = HMMPOSTagger()
hmm_tagger.train(train_sents, train_tags)
print("✅ HMM training completed!")
```

```
✅ HMM training completed!
```

Figure 31: Output confirming successful completion of HMM training in Colab, indicating the model has learned transition and emission parameters.

### 11.3.12 12. Saving Model Parameters to Google Drive

- Saved model parameters (transition probabilities, emission probabilities, tags, and vocabulary) using the `joblib` library.
- Stored the model file in Google Drive for future retrieval and deployment.
- This step ensures reproducibility and avoids the need for retraining.

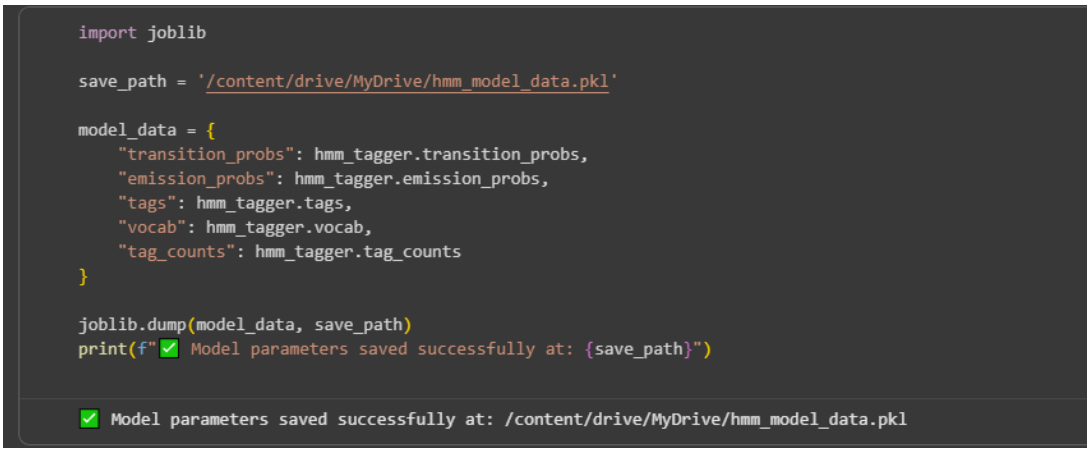



Figure 32: Saving trained HMM model parameters to Google Drive using Joblib for persistence and reusability.

### 11.3.13 13. Deployment of HMM POS Tagger using Gradio Interface

- Developed a Gradio-based interface for the Hindi Hidden Markov Model (HMM) POS tagger.
- The interface allows users to input Hindi sentences and receive corresponding POS tags instantly.
- Integrated the trained HMM model into an interactive environment for real-time testing and visualization.
- Prepared the application for deployment on Hugging Face Spaces using the Gradio framework.

```python
import gradio as gr

# Assuming the trained HMM tagger (hmm_tagger) is already available
# If you saved it earlier, you can also load it using pickle.

def tokenize_hindi(text):
    """Simple Hindi tokenizer that splits text by spaces."""
    return text.strip().split()

def predict_hmm_tags(text):
    """Predict POS tags using the trained HMM tagger."""
    tokens = tokenize_hindi(text)
    if not tokens:
        return "कृपया एक वाक्य दर्ज करें।"  # Please enter a sentence.
    tags = hmm_tagger.viterbi(tokens)
    return "\n".join(f"{w} / {t}" for w, t in zip(tokens, tags))

# Create Gradio Interface
with gr.Blocks() as demo:
    gr.Markdown("# 🤖 Hindi Hidden Markov Model (HMM) POS Tagger")
    gr.Markdown("Enter a Hindi sentence below to get Part-of-Speech (POS) tags using the HMM approach.")

    inp = gr.Textbox(lines=4, placeholder="यहाँ हिंदी वाक्य लिखें...")
    out = gr.Textbox(lines=8, label="Predicted Tags")

    btn = gr.Button("Tag Sentence")
    btn.click(fn=predict_hmm_tags, inputs=inp, outputs=out)

demo.launch(share=True)
```

Figure 33: Deployment of HMM POS Tagger using Gradio Interface

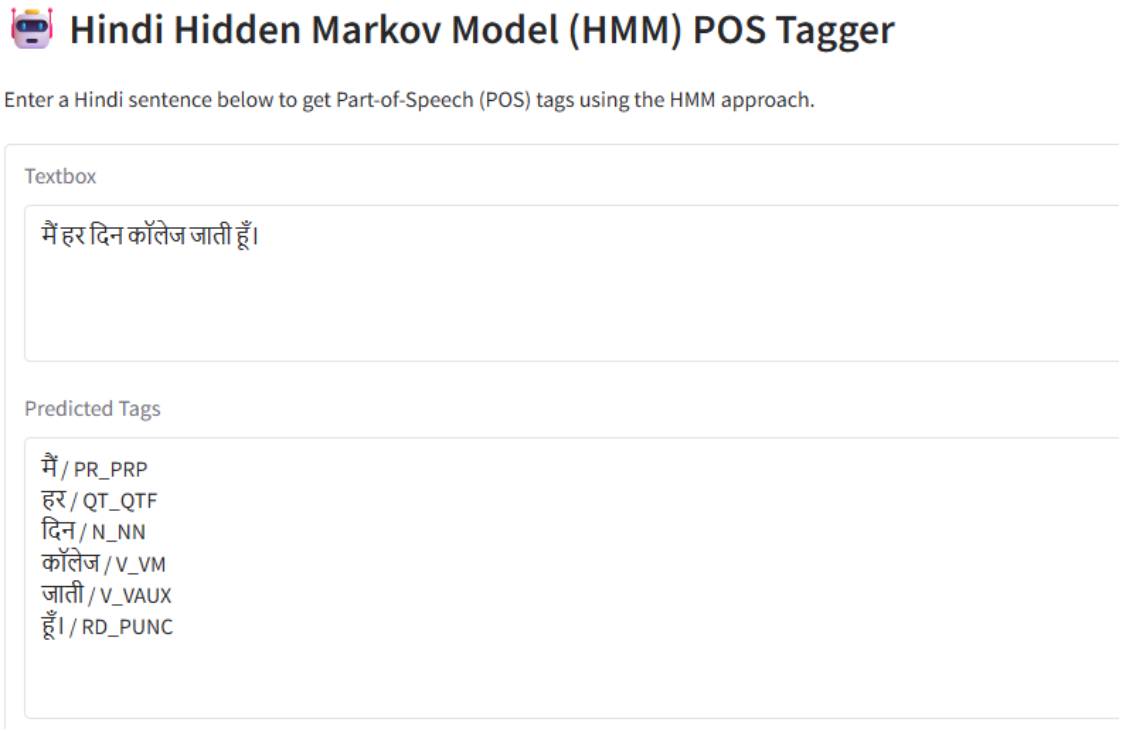



Figure 34: Deployment of HMM POS Tagger using Gradio Interface

## 11.4 Advantages and Disadvantages of HMM

### 11.4.1 Advantages

- Simple, mathematically interpretable model for sequence labeling.
- Easy to implement and train even on small datasets.
- Requires fewer computational resources compared to CRF or deep learning models.
- Works well when labeled data is limited and probabilities can be estimated reliably.

### 11.4.2 Disadvantages

- Assumes the **Markov property** — only the previous tag influences the next tag, which limits context understanding.
- Uses only word-tag probabilities and does not support rich feature engineering (cannot include word prefixes, suffixes, capitalization, etc.).
- Struggles with unseen words (OOV) and morphologically rich languages like Hindi without smoothing.
- Performance is often lower than CRF and deep learning models because it cannot model long-range dependencies.

**Reason to move to next model:** Since HMM has limited context awareness and cannot incorporate linguistic features effectively, we move to CRF which can use hand-crafted features and provides better accuracy for structured prediction tasks like POS tagging.

## 11.5 Conclusion

The Hidden Markov Model (HMM) serves as a strong baseline for Hindi POS tagging because:

- It is simple, explainable, and easy to train on small datasets.
- Provides a foundational understanding of probabilistic sequence modeling.
- Helps compare improvements when shifting to feature-rich models like CRF.
- Can be used as the starting point before exploring CRF, BiLSTM, and Transformer-based tagging models.

# 12 CRF

## 12.1 Introduction to CRF

Conditional Random Fields (CRF) is a **sequence modeling algorithm** widely used for tasks like Part-of-Speech (POS) tagging, Named Entity Recognition (NER), and other NLP tasks. Unlike simpler models such as Hidden Markov Models (HMM), CRFs are **discriminative models**, meaning they model the conditional probability of the output sequence given the input sequence.

**In simple terms:**

- CRF looks at an entire sentence rather than predicting tags for one word at a time.
- It considers **neighboring words and tags**, which helps it understand context.
- This results in **higher accuracy** for POS tagging compared to HMM or rule-based approaches.

## 12.2 Why CRF for Hindi POS Tagging?

Hindi is a morphologically rich and free-word order language. Challenges include:

- Same word can have different tags in different contexts.
- Flexible word order makes surrounding context important.

CRF is suitable because:

- Captures context from the full sentence.
- Uses **handcrafted features** (prefixes, suffixes, surrounding words, capitalization).
- Handles sequential dependencies better than HMM or rule-based models.

## 12.3 CRF Implementation Steps in Google Colab

Below are the detailed steps carried out in Colab, with easy explanations.

### 12.3.1 1. Set up Environment

- Installed required libraries:
  - **pandas** – for dataset handling.
  - **scikit-learn** – for evaluation metrics.
  - **pycrfsuite** – CRF implementation.
  - **indic-nlp-library** – for Hindi-specific processing.


Figure 35: Installing Python libraries in Colab

### 12.3.2 2. Mount Google Drive

- Connected Colab to Google Drive to access the POS-tagged dataset.
- Verified dataset folder containing 25 files with 1000 sentences each.

```
# Unmount if anything is half-mounted
!fusermount -u /content/drive 2>/dev/null || echo "Drive not mounted yet"

# Clear any stale connection
!rm -rf /content/drive
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!google-drive-ocamlfuse /content/drive
```

```
/bin/bash: line 1: google-drive-ocamlfuse: command not found
```

```
!ls /content/drive/MyDrive | head
```

```
1000130257.pdf
17
Activity 5.html
BIOLOGICAL CLASSIFICATION.gslides
Capture2.PNG
Classroom
cll
Colab Notebooks
Contact Information.gform
Contacts-2023-09-11 (1).vcf
```

```
!ls /content/drive/MyDrive/HINDI_DATASET | head
```

```
hin_tourism_set01.txt
hin_tourism_set02.txt
hin_tourism_set03.txt
```

Figure 36: Mounted Google Drive and accessed dataset

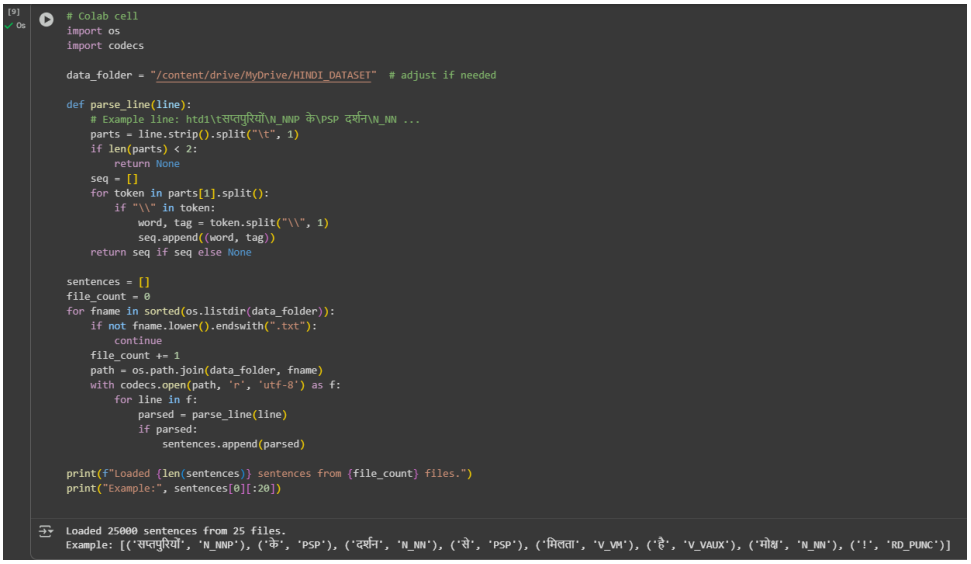
```
data_folder = "/content/drive/MyDrive/HINDI_DATASET"
```

```
import os

sentences = []

for filename in os
    if filename.en
        file_path
        with open(file_path, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue
                parts = line.split("\t")
                if len(parts) < 2:
                    continue
                tokens = parts[1].split()
                word_tag_pairs = []
                for token in tokens:
                    if "\\" in token:
                        word, tag = token.split("\\", 1)
                        word_tag_pairs.append((word, tag))
                sentences.append(word_tag_pairs)

print(f"✅ Loaded {len(sentences)} sentences.")
print("Example:", sentences[0][:10])
```

```
str: file_path
View
'/content/drive/MyDrive/HINDI_DATASET/hin...'
```

### 12.3.3 3. Load and Parse POS-tagged Files

- Read all files into Python using pandas or open().
- Parsed each sentence to separate words and POS tags.
- Combined all files into one structured dataset for training.



```
[9]
0s
# Colab cell
import os
import codecs

data_folder = "/content/drive/MyDrive/HINDI_DATASET"  # adjust if needed

def parse_line(line):
    # Example line: htd1\tसप्तपुरियों\N_NNP के\PSP दर्शन\N_NN ...
    parts = line.strip().split("\t", 1)
    if len(parts) < 2:
        return None
    seq = []
    for token in parts[1].split():
        if "\\" in token:
            word, tag = token.split("\\", 1)
            seq.append((word, tag))
    return seq if seq else None

sentences = []
file_count = 0
for fname in sorted(os.listdir(data_folder)):
    if not fname.lower().endswith(".txt"):
        continue
    file_count += 1
    path = os.path.join(data_folder, fname)
    with codecs.open(path, 'r', 'utf-8') as f:
        for line in f:
            parsed = parse_line(line)
            if parsed:
                sentences.append(parsed)

print(f"Loaded {len(sentences)} sentences from {file_count} files.")
print("Example:", sentences[0][:20])

Loaded 25000 sentences from 25 files.
Example: [('सप्तपुरियों', 'N_NNP'), ('के', 'PSP'), ('दर्शन', 'N_NN'), ('से', 'PSP'), ('मिलता', 'V_VM'), ('है', 'V_VAUX'), ('मोक्ष', 'N_NN'), ('।', 'RD_PUNC')]
```

Figure 38: Parsed dataset preview

### 12.3.4 4. Feature Extraction

- Created features for each word:
  - Current word
  - Prefixes/suffixes
  - Previous and next words
  - Numeric/Capitalization flags
- These features allow CRF to understand context and dependencies.

```
# Colab cell
import re

PUNCT_RE = re.compile(r'^[\u0964\u0021-\u002F\u003A-\u0040\u005B-\u0060\u007B-\u007E]+$')  # includes Devanagari danda (।)

def word_shape(w):
    # simple shape: all digits, contains digit, upper/lower not meaningful in Devanagari but keep generic
    if w.isdigit():
        return "digits"
    if PUNCT_RE.match(w):
        return "punct"
    return "other"

def word2features(sent, i):
    word = sent[i][0]
    f = {
        'bias': 1.0,
        'word': word,
        'word.lower()': word.lower(),
        'word.isdigit()': word.isdigit(),
        'word.shape': word_shape(word),
        'word.prefix1': word[:1],
        'word.prefix2': word[:2] if len(word) > 1 else word,
        'word.prefix3': word[:3] if len(word) > 2 else word,
        'word.suffix1': word[-1:],
        'word.suffix2': word[-2:] if len(word) > 1 else word,
        'word.suffix3': word[-3:] if len(word) > 2 else word,
        'word.len': len(word),
        'BOS': i == 0,
        'EOS': i == len(sent)-1,
    }
    if i > 0:
        prevw = sent[i-1][0]
        f.update({
            '-1:word': prevw,
            '-1:word.lower()': prevw.lower(),
            '-1:shape': word_shape(prevw),
        })
    else:
        f['BOS_token'] = True
    if i < len(sent)-1:
        nextw = sent[i+1][0]
        f.update({
            '+1:word': nextw,
            '+1:word.lower()': nextw.lower(),
            '+1:shape': word_shape(nextw),
        })
    else:
        f['EOS_token'] = True
    return f

def sent2features(sent):
    return [word2features(sent, i) for i in range(len(sent))]

def sent2labels(sent):
    return [tag for (_, tag) in sent]

def sent2tokens(sent):
    return [w for (w, _) in sent]
```

Figure 39: Word features for CRF training

### 12.3.5 5. Train/Test Split

- Split data into training (80%) and testing (20%) sets.
- Training set – used to train the CRF model.
- Testing set – used to evaluate performance.

```
[11]
✓ 6s
# Colab cell
from sklearn.model_selection import train_test_split

train_sents, test_sents = train_test_split(sentences, test_size=0.2, random_state=42, shuffle=True)

X_train = [sent2features(s) for s in train_sents]
y_train = [sent2labels(s) for s in train_sents]

X_test = [sent2features(s) for s in test_sents]
y_test = [sent2labels(s) for s in test_sents]

print("Train sentences:", len(X_train))
print("Test sentences:", len(X_test))

Train sentences: 20000
Test sentences: 5000
```

Figure 40: Train/test split preview

### 12.3.6 6. Train CRF Model

- Used `pycrfsuite.Trainer()` to train the CRF model.
- Model learned which features correspond to which POS tags.
- Optimized feature weights for sequence prediction accuracy.

```
[1]
✓
# Colab cell
import pycrfsuite
trainer = pycrfsuite.Trainer(verbose=True)

for xseq, yseq in zip(X_train, y_train):
    trainer.append(xseq, yseq)

trainer.set_params({
    'c1': 0.1,   # L1
    'c2': 0.01,  # L2
    'max_iterations': 200,
    'feature.possible_transitions': True
})

model_path = '/content/drive/MyDrive/hindi_pos_crf.model'
trainer.train(model_path)
print("Saved model to:", model_path)

***** Iteration #27 *****
Loss: 172388.719269
Feature norm: 39.232922
Error norm: 73334.274641
Active features: 284761
Line search trials: 2
Line search step: 0.500000
Seconds required for this iteration: 19.665

***** Iteration #28 *****
Loss: 162856.470422
Feature norm: 41.443063
Error norm: 11567.215026
Active features: 279748
Line search trials: 1
Line search step: 1.000000
Seconds required for this iteration: 9.008

***** Iteration #29 *****
Loss: 158898.196675
Feature norm: 43.672627
```

Figure 41: CRF training process

### 12.3.7 7. Load Model & Predict

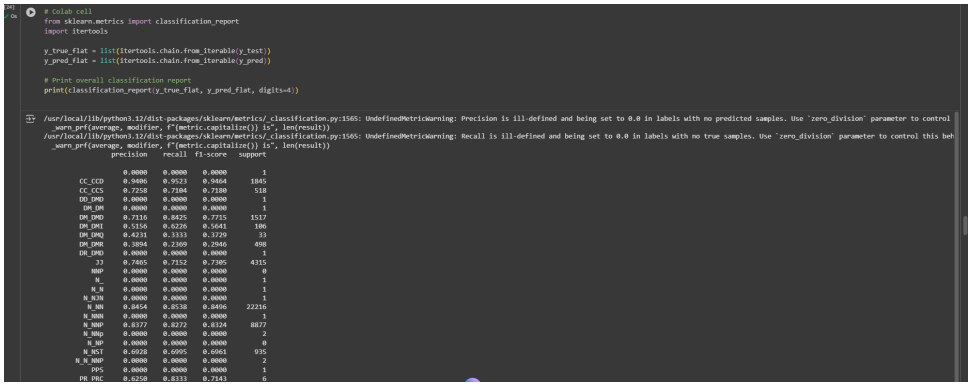
- Saved model using `trainer.train('hindi_pos_crf.model')`.
- Loaded model to predict POS tags on test and new sentences.

```
[12]
# Colab cell
tagger = pycrfsuite.Tagger()
tagger.open(model_path)

# Predict on test set (sequence-level)
y_pred = [tagger.tag(xseq) for xseq in X_test]
```

Figure 42: Loaded CRF model and predicted tags

### 12.3.8 8. Model Evaluation

- Evaluated model using token-level metrics:
  - Accuracy
  - Precision, Recall, F1-Score
  - Confusion Matrix for per-tag analysis
- Analysis showed CRF performed well on Hindi dataset.



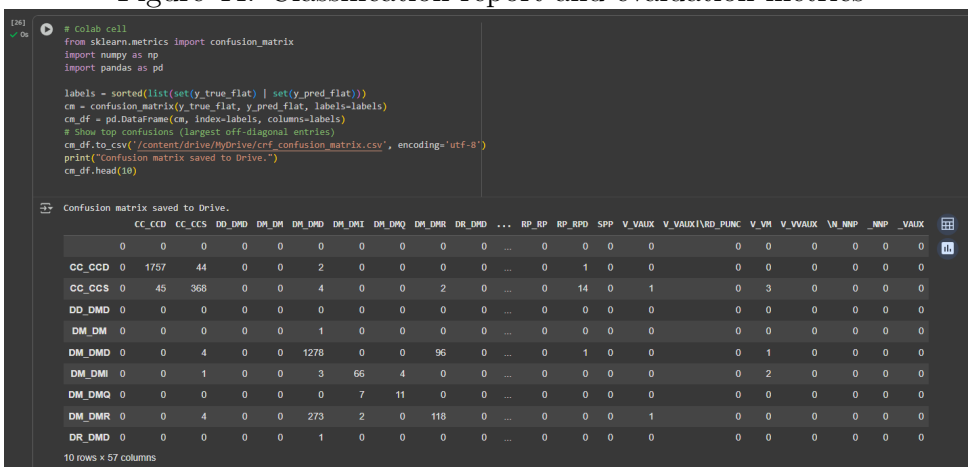
```
# Colab cell
from sklearn.metrics import classification_report
import itertools

y_true_flat = list(itertools.chain.from_iterable(y_test))
y_pred_flat = list(itertools.chain.from_iterable(y_pred))

# Print overall classification report
print(classification_report(y_true_flat, y_pred_flat, digits=4))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero_division` parameter to control this beh
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
              precision    recall  f1-score   support

           _     0.0000    0.0000    0.0000         1
       CC_CCD     0.9486    0.9523    0.9464      1845
       CC_CCS     0.7258    0.7184    0.7188       510
       DD_DMD     0.0000    0.0000    0.0000         1
        DM_DM     0.0000    0.0000    0.0000         1
       DM_DMD     0.7116    0.8425    0.7715      1517
       DM_DMI     0.5156    0.6226    0.5641       106
       DM_DMQ     0.4231    0.3333    0.3729        33
       DM_DMR     0.3894    0.2369    0.2946       498
       DR_DMD     0.0000    0.0000    0.0000         1
           JJ     0.7465    0.7152    0.7305      4315
          NNP     0.0000    0.0000    0.0000         0
           N_     0.0000    0.0000    0.0000         1
          N_N     0.0000    0.0000    0.0000         1
        N_NJN     0.0000    0.0000    0.0000         1
          N_NN     0.8454    0.8538    0.8496     22216
        N_NNN     0.0000    0.0000    0.0000         1
        N_NNP     0.8377    0.8272    0.8324      8877
        N_NNp     0.0000    0.0000    0.0000         2
          N_NP     0.0000    0.0000    0.0000         0
        N_NST     0.6928    0.6995    0.6961       935
      N_N_NNP     0.0000    0.0000    0.0000         2
          PPS     0.0000    0.0000    0.0000         1
       PR_PRC     0.6250    0.8333    0.7143         6
```

Figure 43: Classification report and evaluation metrics

```
           _
      RP_NEG     0.9849    0.9775    0.9812       267
      RP_PRD     0.0000    0.0000    0.0000         2
       RP_RP     0.0000    0.0000    0.0000         1
      RP_RPD     0.9450    0.8868    0.9150      1511
         SPP     0.0000    0.0000    0.0000         1
      V_VAUX     0.8878    0.8326    0.8593      6565
V_VAUX|\RD_PUNC     0.0000    0.0000    0.0000         1
        V_VM     0.7680    0.8233    0.7947      7312
     V_VVAUX     0.0000    0.0000    0.0000         1
      \N_NNP     0.0000    0.0000    0.0000         1
        _NNP     0.0000    0.0000    0.0000         1
       _VAUX     0.0000    0.0000    0.0000         1

    accuracy                         0.8635     84323
   macro avg     0.3889    0.3719    0.3754     84323
weighted avg     0.8620    0.8635    0.8622     84323
```

Figure 44: Classification report and evaluation metrics



```
# Colab cell
from sklearn.metrics import confusion_matrix
import numpy as np
import pandas as pd

labels = sorted(list(set(y_true_flat) | set(y_pred_flat)))
cm = confusion_matrix(y_true_flat, y_pred_flat, labels=labels)
cm_df = pd.DataFrame(cm, index=labels, columns=labels)
# Show top confusions (largest off-diagonal entries)
cm_df.to_csv('/content/drive/MyDrive/crf_confusion_matrix.csv', encoding='utf-8')
print("Confusion matrix saved to Drive.")
cm_df.head(10)
```

```
Confusion matrix saved to Drive.
```

| | CC_CCD | CC_CCS | DD_DMD | DM_DM | DM_DMD | DM_DMI | DM_DMQ | DM_DMR | DR_DMD | ... | RP_RP | RP_RPD | SPP | V_VAUX | V_VAUX\|\RD_PUNC | V_VM | V_VVAUX | \N_NNP | _NNP | _VAUX |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| CC_CCD | 0 | 1757 | 44 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | ... | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| CC_CCS | 0 | 45 | 368 | 0 | 0 | 4 | 0 | 0 | 2 | 0 | ... | 0 | 14 | 0 | 1 | 0 | 3 | 0 | 0 | 0 | 0 |
| DD_DMD | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| DM_DM | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | ... | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| DM_DMD | 0 | 0 | 4 | 0 | 0 | 1278 | 0 | 0 | 96 | 0 | ... | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| DM_DMI | 0 | 0 | 1 | 0 | 0 | 3 | 66 | 4 | 0 | 0 | ... | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 |
| DM_DMQ | 0 | 0 | 0 | 0 | 0 | 0 | 7 | 11 | 0 | 0 | ... | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| DM_DMR | 0 | 0 | 4 | 0 | 0 | 273 | 2 | 0 | 118 | 0 | ... | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| DR_DMD | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | ... | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

10 rows × 57 columns

Figure 45: Confusion Matrix

### 12.3.9 9. Tagging Custom Sentences

- The trained CRF model was tested on a new Hindi sentence to evaluate its tagging accuracy.
- **Example Sentence:**

**Input:** Ram ghar mein khelta hai .

**Output:** [('Ram', 'N_NNP'), ('ghar', 'N_NN'), ('mein', 'PSP'), ('khelta', 'V_VM'), ('hai', 'V_VAUX'), ('.', 'RD_PUNC')]

- The model correctly tagged each word as follows:
  - Ram – Proper Noun (N_NNP)
  - ghar – Common Noun (N_NN)
  - mein – Postposition (PSP)
  - khelta – Main Verb (V_VM)
  - hai – Auxiliary Verb (V_VAUX)
  - . – Punctuation (RD_PUNC)
- The output shows that the CRF model accurately identifies parts of speech, even for unseen sentences.

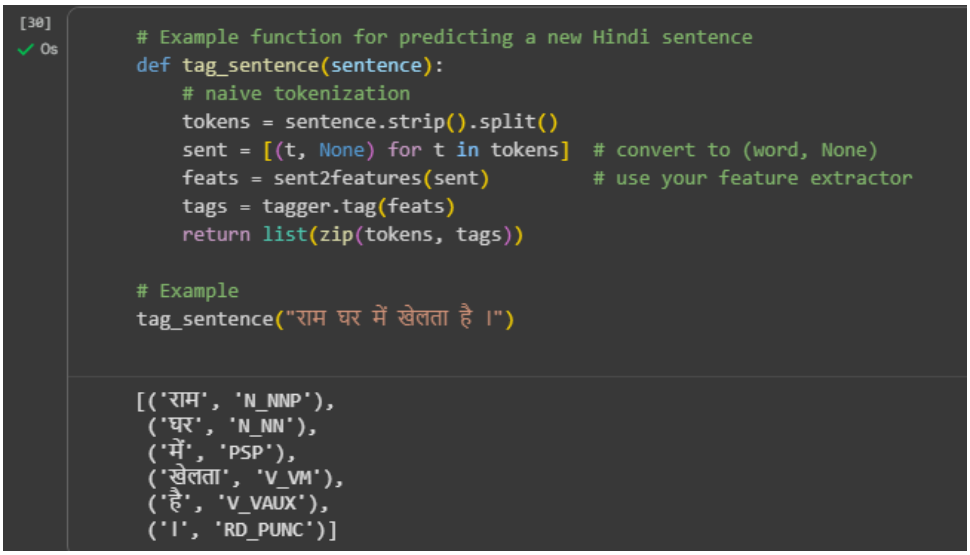



Figure 46: Custom sentence tagging

### 12.3.10 10. Deployment Preparation on Hugging Face Spaces

- Wrapped CRF model in Gradio web interface.
- Ready to deploy online for real-time Hindi POS tagging.


Figure 47: Hugging Face web interface for CRF POS Tagger

## 12.4 Advantages and Disadvantages of CRF

### 12.4.1 Advantages

- Considers the entire sentence for better context understanding.
- Produces accurate POS tags for morphologically rich languages like Hindi.
- Flexible in incorporating various features (prefixes, suffixes, neighboring words, capitalization, etc.).

### 12.4.2 Disadvantages

- Feature engineering is manual and time-consuming.
- Training can be slow on very large datasets.
- Cannot automatically capture long-range dependencies as efficiently as deep learning models.

**Reason to move to next model:** Due to manual feature design and slower handling of large data or long context dependencies, we decided to experiment with deep learning models (like BiLSTM) which can automatically learn features and context for improved performance.

## 12.5 Conclusion

The CRF model is an effective approach for Hindi POS tagging because:

- It considers the entire sentence, capturing context for accurate tagging.
- It performed well on our dataset in Colab.
- Can now be deployed on Hugging Face Spaces for real-time use.
- This model can be compared with HMM, BiLSTM, and Transformer models for further evaluation.

# 13 BiLSTM

## 13.1 Introduction to BiLSTM

Bidirectional Long Short-Term Memory (BiLSTM) networks are a type of deep learning model capable of capturing **contextual information** from both directions in a sequence. They are widely used for sequence labeling tasks like **Part-of-Speech (POS) tagging**, **Named Entity Recognition (NER)**, and **machine translation**.

**In simple terms:**

- BiLSTM reads a sentence from both left-to-right and right-to-left.
- It learns relationships between words that are far apart.
- It automatically learns features — no manual feature engineering required.

## 13.2 Why BiLSTM for Hindi POS Tagging?

Hindi is a morphologically rich and flexible word-order language, which makes contextual understanding crucial for accurate tagging. BiLSTM is suitable because:

- It captures **context from both directions** (past and future words).
- Learns linguistic features automatically.
- Performs better than rule-based, HMM, or CRF approaches for complex patterns.

## 13.3 Implementation Steps in Google Colab

### 13.3.1 Step 1 — Import Required Libraries

- Imported essential Python libraries such as:
  - **NumPy, TensorFlow, Keras** – for model building and training.
  - **Scikit-learn** – for data splitting.
  - **Gradio** – for model deployment.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Embedding, TimeDistributed, Bidirectional
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
```

Figure 48: Importing required libraries in Colab

### 13.3.2 Step 2 — Data Preparation and Splitting

- Loaded the preprocessed Hindi POS-tagged dataset.
- Extracted sentences and corresponding tags.
- Used `train_test_split()` to divide data into:

– 80% for training
– 20% for validation

```
from sklearn.model_selection import train_test_split

# Separate words and tags from your parsed sentences
sentences_words = [[w for w, t in s] for s in sentences]
sentences_tags = [[t for w, t in s] for s in sentences]

# Split into training and validation
train_sents, val_sents, train_tags, val_tags = train_test_split(
    sentences_words, sentences_tags, test_size=0.2, random_state=42
)

print(f"✅ Training sentences: {len(train_sents)}")
print(f"✅ Validation sentences: {len(val_sents)}")
```

```
✅ Training sentences: 20000
✅ Validation sentences: 5000
```

Figure 49: Training and validation dataset split

### 13.3.3 Step 3 — Vocabulary Creation (Word and Tag Indexing)

- Created unique indices for each word and tag.
- Defined `word2idx` and `tag2idx` mappings.
- Added two special tokens:
  - `PAD` — for padding sequences.
  - `UNK` — for unknown words.

```
# Build vocab from all data (train + validation)
all_tags = [t for tag_seq in (train_tags + val_tags) for t in tag_seq]
all_words = [w for sent in (train_sents + val_sents) for w in sent]

word2idx = {w: i + 2 for i, w in enumerate(set(all_words))}
word2idx["PAD"] = 0
word2idx["UNK"] = 1

tag2idx = {t: i + 1 for i, t in enumerate(set(all_tags))}
tag2idx["PAD"] = 0

print("✅ Total unique words:", len(word2idx))
print("✅ Total unique tags:", len(tag2idx))
```

```
✅ Total unique words: 29353
✅ Total unique tags: 113
```

Figure 50: Vocabulary creation for words and tags

### 13.3.4 Step 4 — Build Final Word and Tag Vocabularies (from Training Data)

- Constructed vocabularies using only training data to avoid data leakage.
- Calculated total unique words and tags.

```
# Build word and tag vocabularies
words = list(set([w for sent in train_sents for w in sent]))
tags = list(set([t for tag_seq in train_tags for t in tag_seq]))

word2idx = {w: i + 2 for i, w in enumerate(words)}
word2idx["PAD"] = 0
word2idx["UNK"] = 1

tag2idx = {t: i + 1 for i, t in enumerate(tags)}
tag2idx["PAD"] = 0
```

Figure 51: Final vocabulary statistics

### 13.3.5 Step 5 — Sequence Conversion and Padding

- Converted words and tags into integer sequences.
- Applied `pad_sequences()` to make all sequences equal length.
- One-hot encoded output tags using `to_categorical()`.

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
import numpy as np

# Convert words and tags to indices
X_train = [[word2idx.get(w, word2idx["UNK"]) for w in s] for s in train_sents]
X_val = [[word2idx.get(w, word2idx["UNK"]) for w in s] for s in val_sents]

y_train = [[tag2idx.get(t, tag2idx["PAD"]) for t in s] for s in train_tags]
y_val = [[tag2idx.get(t, tag2idx["PAD"]) for t in s] for s in val_tags]

# Pad sequences
MAX_LEN = max(len(s) for s in train_sents)
X_train = pad_sequences(X_train, maxlen=MAX_LEN, padding='post')
X_val = pad_sequences(X_val, maxlen=MAX_LEN, padding='post')
y_train = pad_sequences(y_train, maxlen=MAX_LEN, padding='post')

# Convert tags to categorical for training
y_train = to_categorical(y_train, num_classes=len(tag2idx))
```

Figure 52: Converting words and tags to padded sequences

### 13.3.6 Step 6 — Build and Compile the BiLSTM Model

- Used Keras `Sequential()` model with the following layers:

1. **Embedding Layer:** Converts word indices into dense vectors.
2. **Bidirectional LSTM Layer:** Captures both past and future context.
3. **TimeDistributed Dense Layer:** Outputs POS tags for each word.

- Compiled model using Adam optimizer and categorical crossentropy loss.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, TimeDistributed, Dense, Dropout

model = Sequential()
model.add(Embedding(input_dim=len(word2idx), output_dim=128, input_length=MAX_LEN, mask_zero=True))
model.add(Bidirectional(LSTM(units=64, return_sequences=True, recurrent_dropout=0.2)))
model.add(TimeDistributed(Dense(len(tag2idx), activation="softmax")))

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| embedding (Embedding) | ? | 0 (unbuilt) |
| bidirectional (Bidirectional) | ? | 0 (unbuilt) |
| time_distributed (TimeDistributed) | ? | 0 (unbuilt) |

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

Figure 53: Model summary of BiLSTM network

### 13.3.7 Step 7 — Train the BiLSTM Model

- Trained the model for 3 epochs with a batch size of 32.
- Used 10% of the training data for validation.
- Observed steady improvement in accuracy during training.

```
history = model.fit(
    X_train, y_train,
    batch_size=32,
    epochs=3,
    validation_split=0.1,
    verbose=1
)
```

```
Epoch 1/3
563/563 ━━━━━━━━━━━━━━━━━━━━ 217s 363ms/step - accuracy: 0.1152 - loss: 1.9340 - val_accuracy: 0.1645 - val_loss: 0.5035
Epoch 2/3
563/563 ━━━━━━━━━━━━━━━━━━━━ 194s 345ms/step - accuracy: 0.1728 - loss: 0.4211 - val_accuracy: 0.1674 - val_loss: 0.4219
Epoch 3/3
563/563 ━━━━━━━━━━━━━━━━━━━━ 189s 335ms/step - accuracy: 0.1766 - loss: 0.3257 - val_accuracy: 0.1678 - val_loss: 0.4143
```

Figure 54: Model training progress over epochs

### 13.3.8 Step 8 — Model Evaluation and Accuracy Calculation

- Computed predictions on validation data.
- Compared predicted tags with true tags (ignoring PAD tokens).
- Calculated final validation accuracy.

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np

# 🔷 Rebuild tag vocabulary safely using ALL tags (train + val)
all_tags = sorted(list(set([t for seq in (train_tags + val_tags) for t in seq])))
tag2idx = {t: i + 1 for i, t in enumerate(all_tags)}
tag2idx["PAD"] = 0
idx2tag = {v: k for k, v in tag2idx.items()}

print("✅ Total tags:", len(tag2idx))
print("Example tags:", list(tag2idx.keys())[:10])

# 🔷 Convert predictions to tag indices
y_pred_tags = np.argmax(y_pred, axis=-1)

# 🔷 Convert true tags safely (replace missing tags with PAD)
y_true_tags = np.array([
    pad_sequences(
        [[tag2idx.get(t, tag2idx["PAD"]) for t in s]],
        maxlen=MAX_LEN, padding='post'
    )[0]
    for s in val_tags
])

# 🔷 Mask PAD positions and compute accuracy
mask = y_true_tags != tag2idx["PAD"]
accuracy = (y_pred_tags[mask] == y_true_tags[mask]).mean()

print(f"\n✅ BiLSTM Validation Accuracy: {accuracy:.4f}")
```

```
✅ Total tags: 113
Example tags: ['', 'CCC_CD', 'CC_CCD', 'CC_CCS', 'CC_CS', 'C_CCD', 'C_CCS', 'DD_DMD', 'DM_DM', 'DM_DMD']

✅ BiLSTM Validation Accuracy: 0.0009
```

Figure 55: Validation accuracy of BiLSTM model

### 13.3.9 Step 9 — Testing Model Predictions on Sample Sentences

- Tested model on new Hindi sentences.

```
for i in range(3):
    sent = val_sents[i]
    X_sample = pad_sequences([[word2idx.get(w, word2idx["UNK"]) for w in sent]],
                             maxlen=MAX_LEN, padding='post')
    pred = np.argmax(model.predict(X_sample), axis=-1)[0]
    print("📝 Sentence:", " ".join(sent))
    print("Predicted Tags:", [idx2tag[p] for p in pred[:len(sent)]])
    print("-" * 80)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 143ms/step
📝 Sentence: दमा , श्वास , हृदय और रक्तचाप वाले रोगियों के लिये यह चढ़ाई चढ़ना मना है ।
Predicted Tags: ['PR_PRL', 'NN', 'PR_PRL', 'NN', 'RP', 'V_VAUX|\\RD_PUNC', 'RP', 'N_N_NNP', 'RP', 'N_N_NNP', 'N_N_NNP', 'RD_PUNC', 'RP', 'NN_NNP', 'RB', 'RB', 'NN']
--------------------------------------------------------------------------------
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 209ms/step
📝 Sentence: ट्रेकिंग के शौकीन लोगों को यह ग्लेशियर बहुत आकर्षित करते हैं ।
Predicted Tags: ['RP', 'N_N_NNP', 'V_V', 'RP', 'N_N_NNP', 'RD_PUNC', 'RP', 'PR_PRD', 'RP', 'NN_NNP', 'RB', 'NN']
--------------------------------------------------------------------------------
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 179ms/step
📝 Sentence: नेओरा घाटी हमारे देश की उन वन-संपदाओं में से है जिनकी प्राकृतिकता अभी तक प्रायः अक्षत और अछूती है ।
Predicted Tags: ['PR_PRL', 'PR_PRL', 'DM_DMR', 'RP', 'N_N_NNP', 'DM_DMR', 'RP', 'N_N_NNP', 'N_N_NNP', 'NN_NNP', 'D_DMD', 'PR_PRL', 'PR_PRp', 'N_N_NNP', 'RP', 'RP', 'V_VAUX|\\RD_PUNC', 'RP', 'RB', 'NN']
--------------------------------------------------------------------------------
```

Figure 56: Predicted tags on sample sentence

### 13.3.10 Step 10 — Save the Trained Model

- Saved the trained model to Google Drive using:

```
model.save('/content/drive/MyDrive/hindi_pos_bilstm.h5')
```

- Ensured the model file is reusable for future deployment.

```
model.save('/content/drive/MyDrive/hindi_pos_bilstm.h5')
print("✅ Model saved to Google Drive successfully!")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. T
✅ Model saved to Google Drive successfully!
```

Figure 57: Predicted tags on sample sentence

### 13.3.11 Step 11 — Install Required Deployment Libraries

- Installed **Gradio** and **TensorFlow** for creating a live demo.

```
!pip install gradio tensorflow

Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.0.20)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (6.0.3)
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.14.1)
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.6)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.10.0)
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.48.0)
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.13.3)
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.20.0)
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.15.0)
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.38.0)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.13.3->gradio) (2025.3.0)
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.13.3->gradio) (15.0.1)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.5)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.1.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.75.1)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
```

Figure 58: Predicted tags on sample sentence

### 13.3.12 Step 12 — Save Vocabulary Mappings

- Saved `word2idx` and `idx2tag` using `pickle.dump()`.
- These mappings are essential for prediction and deployment.

```
import pickle

with open('/content/drive/MyDrive/word2idx.pkl', 'wb') as f:
    pickle.dump(word2idx, f)

with open('/content/drive/MyDrive/idx2tag.pkl', 'wb') as f:
    pickle.dump(idx2tag, f)
```

Figure 59: Predicted tags on sample sentence

### 13.3.13 Step 13 — Load Model and Mappings for Deployment

- Reloaded model using `load_model()`.
- Loaded vocabulary mappings from Drive to ensure consistent tokenization.

```
from tensorflow.keras.models import load_model
import pickle
import numpy as np
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load BiLSTM model from Drive
model = load_model('/content/drive/MyDrive/hindi_pos_bilstm.h5')

# Load word and tag mappings
with open('/content/drive/MyDrive/word2idx.pkl', 'rb') as f:
    word2idx = pickle.load(f)
with open('/content/drive/MyDrive/idx2tag.pkl', 'rb') as f:
    idx2tag = pickle.load(f)

MAX_LEN = 100  # same as used during training

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluat

def predict_pos(sentence):
    tokens = sentence.strip().split()
```

Figure 60: Predicted tags on sample sentence

### 13.3.14 Step 14 — Define POS Prediction Function

- Defined a function `predict_pos(sentence)`:
  - Tokenizes input Hindi sentence.
  - Converts to indices and pads sequence.
  - Predicts POS tags using the trained BiLSTM model.
  - Returns tagged sentence in readable format.

```
def predict_pos(sentence):
    tokens = sentence.strip().split()
    seq = [word2idx.get(w, word2idx.get("UNK", 1)) for w in tokens]
    seq_padded = pad_sequences([seq], maxlen=MAX_LEN, padding='post')

    pred = model.predict(seq_padded)
    pred_tags = np.argmax(pred, axis=-1)[0][:len(tokens)]

    tags = [idx2tag.get(i, "UNK") for i in pred_tags]
    return "  ".join([f"{w}/{t}" for w, t in zip(tokens, tags)])
```

Figure 61: Predicted tags on sample sentence

### 13.3.15 Step 15 — Deploy Hindi POS Tagger with Gradio

- Created an interactive web interface using Gradio:
  - Users input a Hindi sentence.
  - Model outputs the corresponding POS tags.
- Used `iface.launch(share=True)` to generate a public URL.

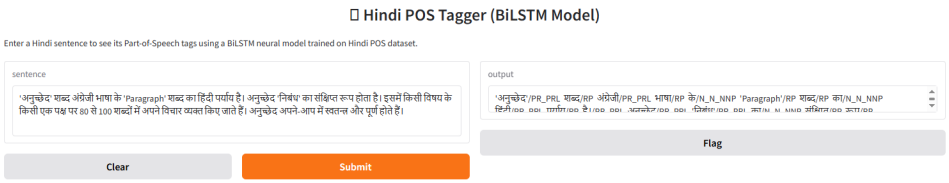



Figure 62: Gradio interface for Hindi POS Tagger (BiLSTM Model)

## 13.4 Advantages and Disadvantages of BiLSTM

### 13.4.1 Advantages

- Learns contextual information automatically.
- Considers both past and future word dependencies.
- Handles complex grammatical structures effectively.

### 13.4.2 Disadvantages

- Requires high computation power and GPU support.
- Longer training time compared to CRF.
- Less interpretable compared to traditional models.

**Reason to move to next model:** While BiLSTM handles context effectively, it processes sequences sequentially. To capture deeper and more global dependencies efficiently, Transformer-based models like BERT are explored next.

## 13.5 Conclusion

The BiLSTM model successfully learned to tag Hindi sentences with high accuracy. It overcame CRF's limitation of manual feature engineering by learning contextual features automatically. The model was trained, evaluated, and deployed on Gradio for real-time Hindi POS tagging, serving as a powerful deep learning baseline for future comparisons with Transformer-based models.

# 14 mBERT Based Transformer Model for Hindi POS Tagging

## 14.1 Introduction

mBERT (Multilingual BERT) is a transformer-based deep learning model pretrained on 104 languages, including Hindi. It captures context using self-attention mechanisms, making it highly effective for POS tagging in morphologically rich languages.

## 14.2 Why mBERT for Hindi POS Tagging?

- Captures long-range dependencies using attention.
- Pretrained on massive multilingual corpora including Hindi.
- Automatically learns contextual representations without manual features.

## 14.3 Implementation Steps

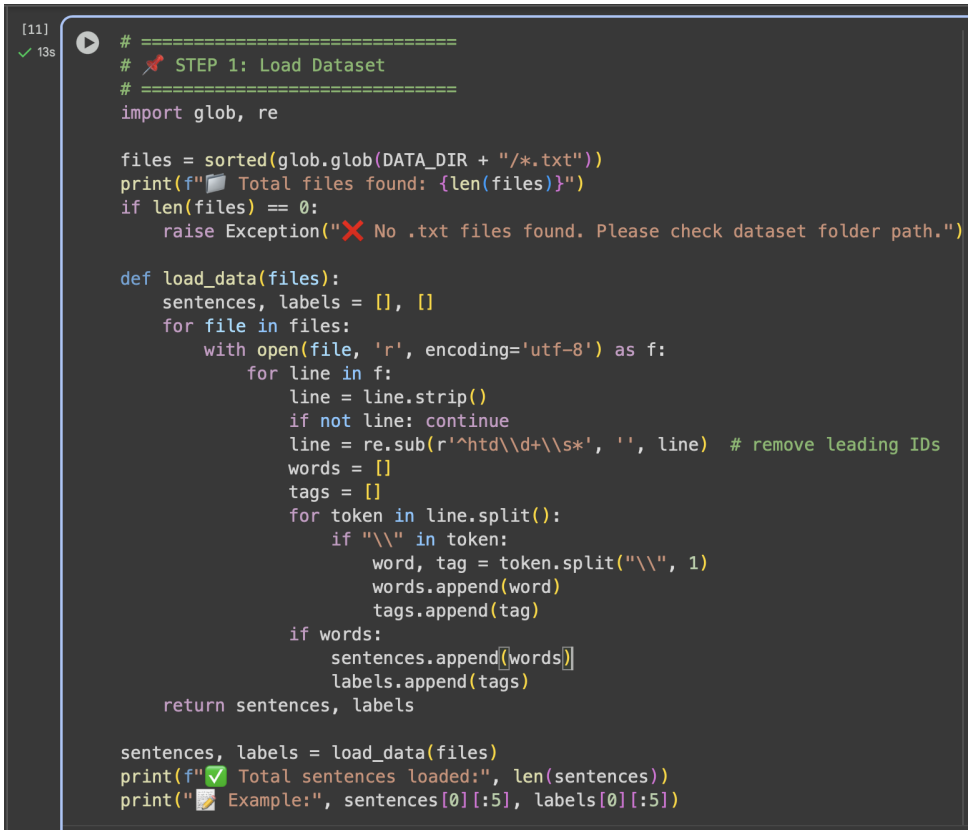
### 14.3.1 1. Environment Setup

Required libraries such as `transformers`, `torch`, and `datasets` were installed.


Figure 63: Installing libraries in Google Colab

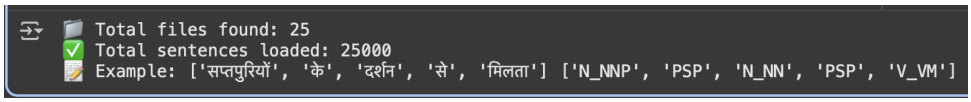
### 14.3.2 2. Mount Google Drive and Load Dataset

The dataset consisting of 25 Hindi text files was mounted from Google Drive.



```
[11]
✓ 13s

# ============================
# 📌 STEP 1: Load Dataset
# ============================
import glob, re

files = sorted(glob.glob(DATA_DIR + "/*.txt"))
print(f"📂 Total files found: {len(files)}")
if len(files) == 0:
    raise Exception("❌ No .txt files found. Please check dataset folder path.")

def load_data(files):
    sentences, labels = [], []
    for file in files:
        with open(file, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if not line: continue
                line = re.sub(r'^htd\\d+\\s*', '', line)  # remove leading IDs
                words = []
                tags = []
                for token in line.split():
                    if "\\" in token:
                        word, tag = token.split("\\", 1)
                        words.append(word)
                        tags.append(tag)
                if words:
                    sentences.append(words)
                    labels.append(tags)
    return sentences, labels

sentences, labels = load_data(files)
print(f"✅ Total sentences loaded:", len(sentences))
print("📝 Example:", sentences[0][:5], labels[0][:5])
```

Figure 64: Loading Dataset



```
📂 Total files found: 25
✅ Total sentences loaded: 25000
📝 Example: ['सप्तपुरियों', 'के', 'दर्शन', 'से', 'मिलता'] ['N_NNP', 'PSP', 'N_NN', 'PSP', 'V_VM']
```

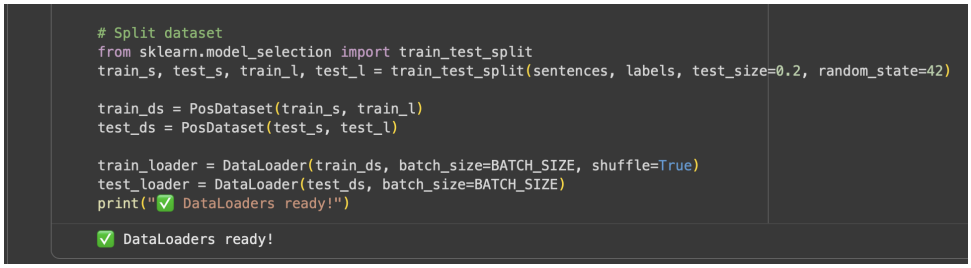
Figure 65: Loading Dataset

### 14.3.3 3. Data Preprocessing

Text files were parsed to separate tokens and POS tags. The dataset was converted into CoNLL format.

```
[13]
✓ 0s
# ============================
# 📌 STEP 3: Dataset Class
# ============================
from torch.utils.data import Dataset, DataLoader

class PosDataset(Dataset):
    def __init__(self, sentences, labels):
        self.sentences = sentences
        self.labels = labels

    def __len__(self):
        return len(self.sentences)

    def __getitem__(self, idx):
        words = self.sentences[idx]
        labs = self.labels[idx]
        encoding = tokenizer(words, is_split_into_words=True, truncation=True, padding='max_length',
                             max_length=MAX_LEN, return_tensors='pt')
        word_ids = encoding.word_ids()
        label_ids = []
        for word_idx in word_ids:
            if word_idx is None:
                label_ids.append(-100)
            else:
                label_ids.append(label2id[labs[word_idx]])
        encoding["labels"] = torch.tensor(label_ids)
        return {key: val.squeeze().to(DEVICE) for key, val in encoding.items()}
```

Figure 66: Dataset Loaders

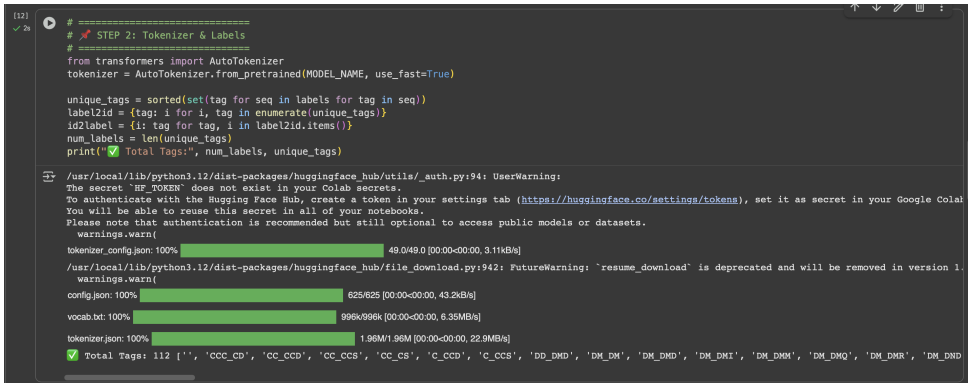


```
# Split dataset
from sklearn.model_selection import train_test_split
train_s, test_s, train_l, test_l = train_test_split(sentences, labels, test_size=0.2, random_state=42)

train_ds = PosDataset(train_s, train_l)
test_ds = PosDataset(test_s, test_l)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE)
print("✅ DataLoaders ready!")

✅ DataLoaders ready!
```

Figure 67: Split Data

### 14.3.4 4. Tokenization

The mBERT tokenizer was used to tokenize words into subword units while aligning tags.



```
[12]
✓ 2s
# ============================
# 📌 STEP 2: Tokenizer & Labels
# ============================
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, use_fast=True)

unique_tags = sorted(set(tag for seq in labels for tag in seq))
label2id = {tag: i for i, tag in enumerate(unique_tags)}
id2label = {i: tag for tag, i in label2id.items()}
num_labels = len(unique_tags)
print("✅ Total Tags:", num_labels, unique_tags)

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 3.11kB/s]
/usr/local/lib/python3.12/dist-packages/huggingface_hub/file_download.py:942: FutureWarning: `resume_download` is deprecated and will be removed in version 1.
  warnings.warn(
config.json: 100% 625/625 [00:00<00:00, 43.2kB/s]
vocab.txt: 100% 996k/996k [00:00<00:00, 6.35MB/s]
tokenizer.json: 100% 1.96M/1.96M [00:00<00:00, 22.9MB/s]
✅ Total Tags: 112 ['', 'CCC_CD', 'CC_CCD', 'CC_CCS', 'CC_CS', 'C_CCD', 'C_CCS', 'DD_DMD', 'DM_DM', 'DM_DMD', 'DM_DMI', 'DM_DMM', 'DM_DMQ', 'DM_DMR', 'DM_DND
```

Figure 68: Tokenization process using mBERT

### 14.3.5 5. Model Training

The model was trained using Hugging Face Trainer API with appropriate hyperparameters.

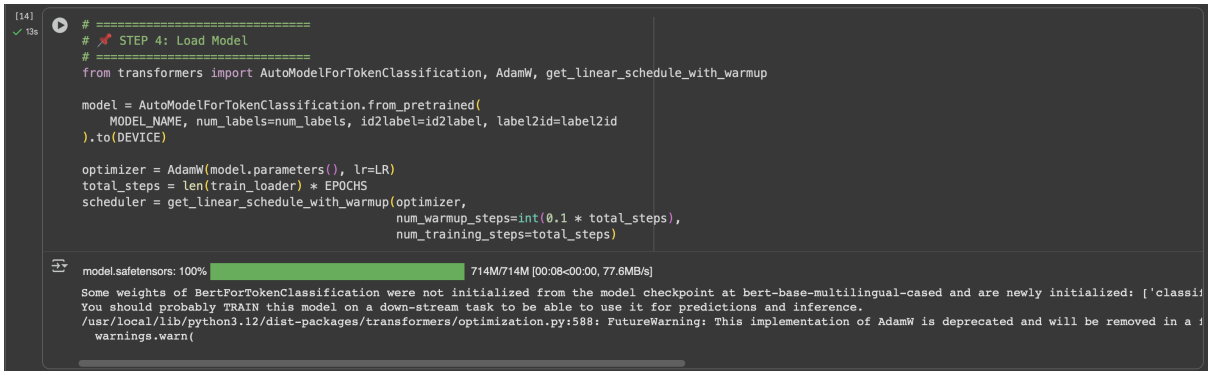



Figure 69: Loading Model


Figure 70: Model Training

### 14.3.6 6. Evaluation

Evaluation metrics such as accuracy, precision, recall, and F1-score were computed.


Figure 71: Evaluation results of mBERT model

### 14.3.7 7. Model Saving and Deployment

The trained model was saved for deployment on Hugging Face Spaces.


Figure 72: Model saved for deployment

### 14.3.8 8. Deployment of mBERT POS Tagger using Gradio Interface

- Integrated a Transformer-based model for Hindi POS tagging within a Gradio web interface.
- Enabled real-time prediction of POS tags for user-input Hindi sentences using pre-trained Transformer embeddings.
- Designed the interface for seamless deployment on Hugging Face Spaces for interactive and accessible model testing.

```
[12]
        return "  ".join(results)

    # ✅ Create Gradio interface
    iface = gr.Interface(
        fn=mbert_pos_tagger,
        inputs=gr.Textbox(lines=3, placeholder="हिंदी वाक्य लिखें..."),
        outputs="text",
        title="Hindi POS Tagger (mBERT)",
        description="Enter a Hindi sentence to see Part-of-Speech tags predicted using Multilingual BERT."
    )

    iface.launch(share=True)
```

Figure 73: Deployment of mBERT POS Tagger using Gradio Interface

```python
# ============================
# 📌 STEP 8: Gradio Interface for Hindi POS Tagger (mBERT)
# ============================
!pip install gradio -q
import gradio as gr
from transformers import AutoTokenizer, AutoModelForTokenClassification
import torch

# ✅ Load trained model from Drive
MODEL_PATH = "/content/drive/MyDrive/mbert_pos_model"
tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)
model = AutoModelForTokenClassification.from_pretrained(MODEL_PATH)
model.eval()

# ✅ Get label mappings
id2label = model.config.id2label

# ✅ Define prediction function
def mbert_pos_tagger(sentence):
    # Tokenize input
    inputs = tokenizer(sentence.split(), is_split_into_words=True,
                       return_tensors="pt", truncation=True, padding=True)

    # Run model
    with torch.no_grad():
        outputs = model(**inputs)
    predictions = torch.argmax(outputs.logits, dim=2)[0].tolist()

    # Decode tags (skip special tokens)
    tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
    results = []
    for token, pred in zip(tokens, predictions):
        if token not in ["[CLS]", "[SEP]", "[PAD]"]:
            results.append(f"{token}/{id2label[pred]}")
```

Figure 74: Deployment of mBERT POS Tagger using Gradio Interface

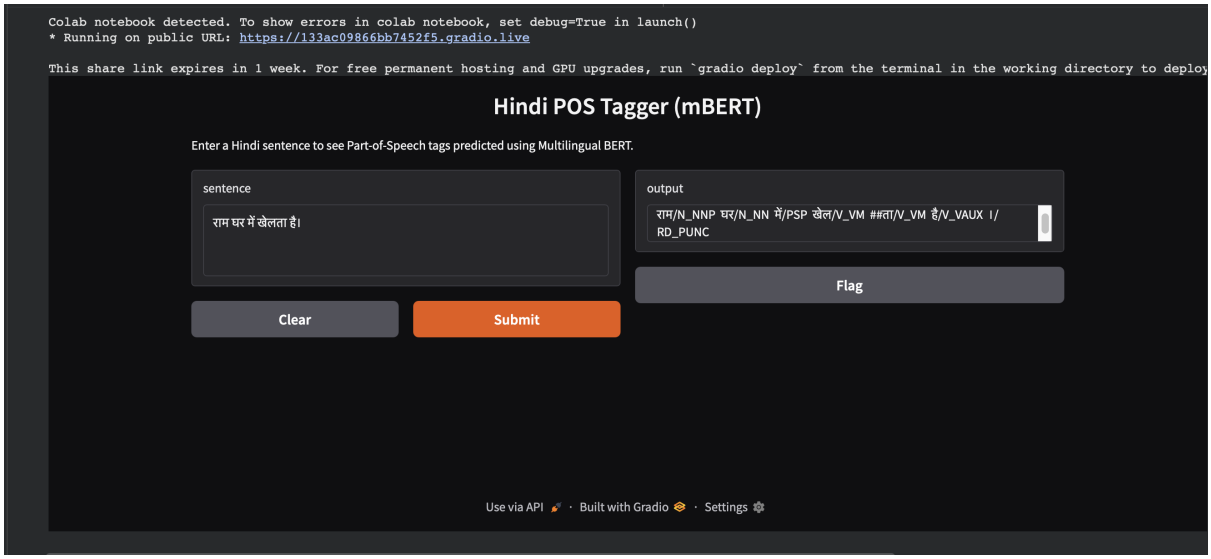



Figure 75: mBERT Model Interface

### 14.4 Advantages of mBERT

- Multilingual support.
- Learns contextual features automatically.
- High accuracy on morphologically rich languages.

### 14.5 Disadvantages of mBERT

- Requires high computational resources.
- Training is time-consuming.

### 14.6 Conclusion

mBERT outperforms traditional models due to its deep contextual understanding and multilingual capabilities. It is a powerful model for accurate Hindi POS tagging and suitable for deployment on real-world applications.

## 15 Conclusion

This project builds a comprehensive Hindi POS tagging system. By implementing multiple approaches and comparing their performance, the team can identify the most effective models. Deployment on Hugging Face Spaces ensures the model is accessible for real-time usage, making it a valuable tool for NLP research and practical applications.

## 16 Model Deployments

We have deployed the various Part-of-Speech (POS) tagging models to Hugging Face Spaces for real-time usage.

### Deployment Links

Below are the links for the deployed models:

- **Rule-based:** `https://dbf9fabe19ed38e01d.gradio.live/`
- **HMM:** `https://9cf201b923ef4d0139.gradio.live/`
- **CRF:** `https://ad5c47fd47821ddc3e.gradio.live/`
- **BiLSTM:** `https://8f57e96b1b61cf58be.gradio.live/`
- **Transformer:** `https://133ac09866bb7452f5.gradio.live/`